AIMHRA — Complete Technical Documentation, Code Walkthrough & Viva Preparation Guide

AIMHRA — Complete Technical Documentation, Code Walkthrough & Viva Preparation Guide Derived entirely from the source code at d:\project\AIMHRA\AIMHRA\v1. Nothing below is taken from documentation alone; every claim is traceable to a file. Where something could not be found, it is explicitly marked "Not found in the inspected codebase."

1. Executive Summary

AIMHRA (AI-based Maternal Health Risk Analysis and Decision-Support System, per backend/config/settings.py line 3 and SPECTACULAR_SETTINGS in the same file) is a full-stack clinical decision-support web application with:

- A Django/DRF REST backend (backend/) serving JWT-authenticated APIs for 9 Django apps: core, audit, accounts, patients, assessments (with a rules/ sub-package), mlcore, kb, chat, reports.
- A React 19 + Vite SPA (frontend/) with role-based routing for two staff roles: HEALTHCARE_WORKER and ADMIN.
- An ML layer that trains Random Forest and XGBoost classifiers on Mathernal_Risk.csv (6,103 raw rows → 6,057 after cleaning) over 8 features, serves the "production" model through a thread-safe registry, and attaches SHAP TreeExplainer explanations to every prediction.
- A deterministic, versioned rule engine (assessments/rules/rules.json, version v1, 21 rules, 4 categories) that runs alongside the model and outranks everything (ML, RAG, LLM) for emergencies.
- A RAG knowledge base (kb app): document ingestion (PDF/TXT/MD), 900-char/150-overlap chunking, TF-IDF (default, offline) or OpenAI text-embedding-3-small embeddings, cosine-similarity retrieval over vectors stored on DocumentChunk rows (default) or an optional Qdrant collection.
- An LLM chat assistant (chat app) grounded in RAG context, with a strict non-diagnosis system prompt, emergency keyword bypass, and graceful fallbacks.
- PDF report generation with ReportLab (reports app).
- Audit logging (audit app) for 19 action types with key redaction.

The working tree also documents an in-progress role refactor: PATIENT was removed as an application role (git status shows deleted frontend/src/pages/patient/* pages, deleted Register.jsx, new 0002 migrations in accounts, patients, and audit). Patients are now standalone clinical records, not login accounts.

2. What AIMHRA Does

Purpose (in simple terms): A healthcare worker logs in, maintains maternal patient records, enters a set of clinical measurements at each visit, and the system:

1. Runs a machine-learning model that classifies the patient as "low risk" / "mid risk" / "high risk" with per-class probabilities.
2. Explains the prediction with SHAP (which features pushed the prediction up or down).
3. Runs independent clinical safety rules over the same inputs (e.g., systolic BP ≥ 160 → EMERGENCY) and creates alerts.
4. Stores everything so a longitudinal history and trend (improving / stable / increasing) can be shown.
5. Can generate a PDF report for any assessment.
6. Provides a RAG-grounded AI assistant per patient that answers educational questions and escalates emergency language deterministically.

Problem it solves: making maternal-risk screening explainable, auditable, and safety-guarded, while never pretending to replace a clinician. Disclaimers are baked into every layer (assessment responses, SHAP output, trends, chat, PDF).

Intended users (from accounts/models.py):

- HEALTHCARE_WORKER — manages patient records, assessments, alerts, reports.
- ADMIN — manages users, models, knowledge base, audit logs; sees all patient data.
- Patients are explicitly not users: patients/models.py docstring: "Patients are NOT application users: they never log in, register, or hold a password."

End-to-end flow (verified in code):

User Login (frontend/src/pages/auth/Login.jsx)

```
    -> POST /api/auth/login/ (accounts/views.py LoginView)
```

JWT access + refresh issued; refresh blacklisted at logout

```
    ->
Role Dashboard (App.jsx RoleDashboard -> pages/hcw/Dashboard.jsx or pages/admin/Dashboard.jsx)
    ->
Patient selection (pages/hcw/Patients.jsx -> GET /api/patients/, scope = created union assigned)
    ->
```

Patient detail workspace (pages/hcw/PatientDetail.jsx, 7 tabs)

```
    ->
```

New Assessment (components/patient/AssessmentPanel.jsx)

```
    -> client validation (min/max, dia <= sys)
```

POST /api/assessments/ (assessments/views.py AssessmentListCreateView.create)

```
    -> AssessmentSerializer validation (ranges from mlcore/constants.PLAUSIBILITY_RANGES)
    -> create_assessment() (assessments/services.py)
    -> mlcore.registry.predict(): feature vector -> Preprocessor (median impute) -> model.predict_proba
    -> risk class = argmax(probabilities)
    -> mlcore.explain.explain_prediction(): SHAP TreeExplainer top-8 contributions
    -> assessments.rules.evaluate(): rules.json -> category NORMAL/ATTENTION/HIGH_CONCERN/EMERGENCY
    -> transaction.atomic(): Assessment + Prediction (+ Alert if != NORMAL) rows
    -> audit events (ASSESSMENT_CREATED, PREDICTION, RULE_ESCALATION)
    -> trend computed over prediction series (assessments/trends.compute_trend)
    -> JSON response -> PredictionResult UI (risk badge, probability bars, SHAP bars, rules, disclaimer)
    ->
History (HistoryPanel -> /api/assessments/risk-history/) & Trend (RiskTrendChart)
    ->
Report (reportService.generate -> reports/services.generate_report -> ReportLab PDF -> download)
    ->
AI Assistant (ChatPanel -> /api/chat/sessions/{id}/send/ -> keyword detect -> rules -> RAG -> LLM/fallback)
```

3. Complete Technology Stack

Backend (from backend/requirements.txt)

Technology	Where used	Why used	If removed…
Python 3.12 (venv has .cpython-312.pyc files)	everywhere	The project language	—
Django >=5.1,<5.2	backend/config/settings.py, all apps	ORM, auth, admin, migrations, URL routing, templates (admin only)	No server, no ORM, no auth
Django REST Framework	REST_FRAMEWORK settings; every view	Serializers, views, permissions, pagination, exception handling	Hand-rolled JSON endpoints
djangorestframework-simplejwt (+ token_blacklist)	accounts/serializers.py, SIMPLE_JWT settings, LogoutView	Stateless JWT auth; refresh-token blacklisting for real logout	Sessions or insecure tokens
django-cors-headers	MIDDLEWARE, CORS_ALLOWED_ORIGINS	Lets the Vite dev origin (5173) call the API	Browser blocks cross-origin calls
drf-spectacular	config/urls.py (/api/schema/, /api/docs/)	Auto OpenAPI schema + Swagger UI	No interactive API docs
python-dotenv	settings.load_dotenv(BASE_DIR / ".env")	Loads env vars for secrets/config	Secrets hardcoded
numpy	mlcore/*, kb/services/store.py	Vector math, feature matrices, cosine similarity	No ML or vector math
pandas	mlcore/dataset.py	CSV loading, cleaning, dedup, class stats	No data pipeline
scikit-learn	mlcore/training.py (RandomForestClassifier, train_test_split), mlcore/evaluation.py, kb/services/embeddings.py (TfidfVectorizer)	RF model, splits, metrics, TF-IDF embeddings	Half the ML stack
xgboost	mlcore/training.py _make_xgboost()	Gradient-boosted trees, second candidate model	Only one model family
shap	mlcore/explain.py	TreeExplainer feature attributions (explainability)	Predictions become black boxes
joblib	mlcore/training.py (dump), mlcore/registry.py (load)	Persist/load model bundles (ml_artifacts/*.joblib)	No trained-model persistence
reportlab	reports/services.py	Server-side PDF generation	No printable reports
openai	chat/llm.py, kb/services/embeddings.py	OpenAI-compatible Chat Completions + embeddings client	No LLM/RAG generation (fallbacks exist)
requests	listed in requirements	Not imported anywhere in the inspected code. Unused dependency.	Nothing breaks

Installed in venv but missing from requirements.txt (found by import tracing): pypdf (used by kb/services/ingestion.py extract_text() — PDF knowledge ingestion silently breaks on a fresh install), dj_database_url (imported by settings.py only when DATABASE_URL starts with postgres://), qdrant_client (optional, code catches ImportError and falls back — correctly documented in kb/services/store.py).

Frontend (from frontend/package.json)

Package	Where used	Why
react 19 / react-dom 19	all of src/	Component UI library
react-router-dom 7	main.jsx, App.jsx, ProtectedRoute.jsx, Layout.jsx, all pages	Client routing + nested layouts + route guards
axios	src/api/client.js only	HTTP client with interceptors (JWT attach, 401 refresh-retry)
recharts 2.15	src/charts/index.jsx	LineChart (trend), PieChart (distribution), BarChart (SHAP/importance/metrics)
vite 8 + @vitejs/plugin-react	vite.config.js	Dev server (port 5173, proxy /api + /media -> localhost:8000), bundler
oxlint	npm run lint, .oxlintrc.json	Linting (react/hooks rules)
@types/react, @types/react-dom	dev only	Type hints for editors (project is plain JS)

Not used (verified): Redux, Zustand, Tailwind, component libraries, Lucide, LangChain, LangGraph, Gemini/Groq. Icons are Unicode characters; styling is hand-written CSS.

- LLM: any OpenAI-compatible Chat Completions API; default gpt-4o-mini at https://api.openai.com/v1 (settings.py lines 172–175, chat/llm.py). No

Gemini/Groq/LangChain anywhere.

- Embeddings: default local TF-IDF (TfidfVectorizer(max_features=5000, stop_words="english") in kb/services/embeddings.py), optional OpenAI text-embedding-3-small.
- Vector store: default local — embeddings stored as JSON on DocumentChunk.embedding, ranked by cosine similarity in numpy (kb/services/store.py); optional Qdrant collection aimhra_kb if VECTOR_BACKEND=qdrant.
- RAG: retrieval in kb/services/rag.py (top-5, min score 0.05), generation in chat/services.py + chat/llm.py.

AI/RAG stack (verified in code)

4. Overall System Architecture

React SPA (Vite, port 5173)

```
   |  axios (src/api/client.js) — JWT in Authorization header
   |  dev proxy: /api -> http://localhost:8000 (vite.config.js)
   v
```

Django (config/urls.py)

```
   v
```

DRF layer

```
   |- JWTAuthentication (DEFAULT_AUTHENTICATION_CLASSES)
   |- IsAuthenticated default permission (settings.py:122)
   |- core.permissions.IsHealthcareWorker / IsAdmin / IsAdminOrReadOnly
   |- core.pagination.StandardPagination (page_size 20)
   `- core.exceptions.structured_exception_handler -> {success, error{code,message,details}}
   v
Views -> Serializers (validation) -> services/selectors (business logic)
   v
Django ORM -> SQLite (backend/db.sqlite3)   [DATABASE_URL can switch to Postgres/MySQL]
   v
```

Domain engines:

```
   |- mlcore.registry -> joblib bundle -> RF/XGB predict_proba -> SHAP
   |- assessments.rules -> rules.json (deterministic, outranks everything)
   |- kb.services (RAG retrieval) -> chat.llm (OpenAI-compatible generation)
   `- reports.services -> ReportLab -> media/reports/*.pdf
```

- Serializer layer = input shape + range validation (assessments/serializers.py validates against the same PLAUSIBILITY_RANGES used to clean training data — a deliberate train/serve consistency decision, stated in its docstring).
- View layer = HTTP orchestration + access control (can_access_patient checks) + response envelope.
- Service layer = transactional workflows (create_assessment, generate_report, respond, index_document) — no HTTP knowledge.
- Selectors = reusable query/scope logic (patients/selectors.py).
- Registry = the only place that touches model artifacts.
- Frontend never calls axios directly from components; everything goes through src/api/client.js helpers and src/services/auth.js domain services.

Layer responsibilities:

5. Complete Project Folder Structure (actual)

```
v1/
├── README.md                       # nearly empty: just "# AIMHRA"
├── .gitignore
├── Mathernal_Risk.csv              # duplicate copy of the dataset (root)
├── pv1.zip                         # unexplained archive (see §38/43)
├── scripts/
│   └── dataset_profile.py          # stdlib-only CSV profiler (uses root CSV)
├── backend/
│   ├── manage.py                   # Django CLI entry (DJANGO_SETTINGS_MODULE=config.settings)
│   ├── requirements.txt
│   ├── .env.example                # contains a real-looking API key (see §36)
│   ├── db.sqlite3                  # dev database (gitignored)
│   ├── admin.txt                   # empty file
│   ├── full_test_out.txt           # stray test output (untracked)
│   ├── visualization.ipynb         # untracked notebook
│   ├── config/                     # settings.py, urls.py, wsgi.py, asgi.py
│   ├── core/                       # permissions, exceptions, middleware, pagination, responses
│   ├── audit/                      # AuditLog model, log_event service, admin-only list API
│   ├── accounts/                   # User(AbstractUser) + role, JWT login/refresh/logout, user mgmt
│   │   └── migrations/0001, 0002 (PATIENT role removal)
│   ├── patients/                   # PatientProfile, PatientAssignment, selectors, admin_urls (stats)
│   │   └── migrations/0001, 0002 (standalone records refactor)
│   ├── assessments/                # Assessment, Prediction, Alert; services, trends, urls
│   │   ├── rules/                  # engine.py, keywords.py, rules.json, __init__.py
│   │   └── migrations/0001
│   ├── mlcore/                     # ML: constants, dataset, preprocessing, training, evaluation,
│   │   │                           #   registry, explain, models(ModelVersion), views, tests
│   │   ├── management/commands/    # train_models, inspect_dataset, generate_synthetic
│   │   └── migrations/0001
│   ├── kb/                         # RAG: KnowledgeDocument, DocumentChunk, upload/reindex/retrieve APIs
│   │   └── services/               # ingestion, indexing, embeddings, store, rag
│   ├── chat/                       # ChatSession, ChatMessage, services (safety pipeline), llm.py
│   ├── reports/                    # Report model, ReportLab services, list/generate/download APIs
```

```
|   ├── data/                     # Mathernal_Risk.csv, dataset_report.json
|   ├── ml_artifacts/             # randomforest_full/quick.joblib, xgboost_full/quick.joblib
|   ├── media/                    # kb/documents/, reports/*.pdf (generated)
|   ├── logs/                     # aimhra.log (RotatingFileHandler)
|   └── scripts/smoke_test.py     # end-to-end workflow smoke test
└── frontend/
    ├── index.html, package.json, vite.config.js, .env.example, .oxlintrc.json
    ├── public/ (favicon.svg, icons.svg)
    └── src/
        ├── main.jsx, App.jsx, styles.css
        ├── api/client.js          # axios instance, JWT interceptors, envelope unwrap
        ├── context/AuthContext.jsx
        ├── components/            # Layout, ProtectedRoute, ui (badges/states), patient/ (7 panels)
        ├── charts/index.jsx       # Recharts wrappers
        ├── constants/clinical.js  # blood-group options
        ├── pages/auth/Login.jsx
        ├── pages/hcw/             # Dashboard, Patients, PatientNew, PatientDetail, Alerts, Reports
        ├── pages/admin/           # Dashboard, Users, Models, Knowledge, Audit
        ├── services/auth.js       # ALL endpoint wrappers (auth, patient, assessment, model, chat, kb, report, admin)
        └── utils/reportDownload.js
```

Deleted in the working tree (role refactor): frontend/src/pages/auth/Register.jsx and the entire frontend/src/pages/patient/ folder (Assessment/Chat/Dashboard/History/Profile/Reports/Settings).

6. Backend Architecture

6.1 backend/config/settings.py

- Loads .env via load_dotenv; SECRET_KEY/DEBUG/ALLOWED_HOSTS from env with dev-safe defaults (testserver host auto-added when DEBUG).
- INSTALLED_APPS: Django contribs + rest_framework, rest_framework_simplejwt (+token_blacklist), corsheaders, drf_spectacular, then core, audit, accounts, patients, assessments, mlcore, kb, chat, reports.
- MIDDLEWARE: CorsMiddleware first, standard Django stack, plus custom core.middleware.StructuredErrorMiddleware last.
- AUTH_USER_MODEL = "accounts.User".
- REST_FRAMEWORK: JWTAuthentication default, IsAuthenticated default permission, StandardPagination (20), custom exception handler, spectacular schema.
- Database: SQLite default; manual DATABASE_URL parsing — dj_database_url for Postgres (import guarded), manual urlparse config for MySQL.
- SIMPLE_JWT: access 30 min, refresh 7 days (env-overridable), custom obtain/refresh serializers from accounts.serializers.
- CORS_ALLOWED_ORIGINS from env (5173 dev origins).
- ML config: DATASET_PATH (default data/Mathernal_Risk.csv), ML_ARTIFACTS_DIR, MODEL_SELECTION_CRITERION (default high_risk_recall_then_f1).
- LLM/RAG config: LLM_API_KEY/_BASE_URL/_MODEL/_TIMEOUT_SECONDS, EMBEDDING_BACKEND (local), VECTOR_BACKEND (local), VECTOR_DB_URL/_API_KEY.
- Email config for password reset (console backend by default).
- Logging: rotating file logs/aimhra.log, INFO root; explicit note: "structured, avoids sensitive medical payloads".

6.2 backend/config/urls.py

```
Routes: admin/, api/schema/, api/docs/, then api/auth/ -> accounts, api/patients/, api/assessments/, api/models/, api/rag/, api/chat/, api/reports/,
api/audit/, api/admin/ -> patients.admin_urls. Serves media in DEBUG.
```

6.3 backend/core/* (shared infrastructure)

- permissions.py: IsHealthcareWorker (role in HCW union ADMIN), IsAdmin, IsAdminOrReadOnly.
- exceptions.py: ApiError (raise-anywhere structured error), ERROR_CODE_BY_STATUS map, flatten_error_messages (turns nested DRF validation detail into one readable message), structured_exception_handler (rolls back, maps codes, never leaks stack traces; unhandled -> 500 INTERNAL_ERROR logged).
- middleware.py: StructuredErrorMiddleware converts Django's HTML 404/500 on /api/ paths into the JSON envelope; process_exception adds debug_trace only when DEBUG.
- pagination.py: StandardPagination -> {success, data:{count,page,pages,results}}.
- responses.py: ok(data) success envelope helper.
- apps.py: trivial AppConfig.

6.4 Uniform response contract

Every endpoint answers {"success": true, "data": ...} or {"success": false, "error": {code, message, details}}. The frontend (unwrap(), apiError()) is built around this exact contract.

7. Frontend Architecture

- Entry main.jsx: ReactDOM.createRoot -> StrictMode -> BrowserRouter -> AuthProvider -> App.
- State management: React useState/useCallback/useEffect + one Context (AuthContext) for the authenticated user + localStorage (aimhra_access, aimhra_refresh keys, src/api/client.js storage object). No Redux/Zustand (verified).
- API client src/api/client.js:
- Request interceptor attaches Authorization: Bearer <access>.

- Response interceptor: on 401 with a refresh token and not already retried -> POST /auth/refresh/ (single-flight via shared refreshing promise) -> retry original request; on failure clears tokens and dispatches window event aimhra:logout, which AuthContext listens to in order to clear the user.
- unwrap() strips the {success,data} envelope; apiError() normalizes backend/network errors into {code,message,details}.
- api.get/post/patch/put/delete + getRaw (blob) — components never import axios directly.
- Routing App.jsx: /login public; / redirects to /app/dashboard; /app wraps ProtectedRoute + Layout (sidebar per role via NAV_BY_ROLE); nested routes with role arrays; RoleDashboard switches HCW vs Admin dashboard; wildcard redirects to dashboard.
- Rendering: route component -> guard (booting -> Loading; no user -> /login; role not in allowlist -> /login; wrong role -> own dashboard) -> page -> data via services -> conditional Loading/ErrorState/EmptyState/content.

8. Database Architecture

Models (all fields verified)

accounts.User (accounts/models.py) — extends AbstractUser; adds role (choices HCW/ADMIN, default HCW), phone, full_name. save() derives is_staff = (role == ADMIN). Property is_admin_role.

> patients.PatientProfile — standalone record: created_by FK->User (SET_NULL, related patients_created), is_active (archive flag), archived_at, demographics (full_name, email, phone, address), clinical (date_of_birth, blood_group, gestational_week_at_registration, estimated_due_date, gravidity, parity, medical_history_notes, allergies, emergency_contact_name/phone, obstetric_history_notes, current_medications, additional_notes), patient_code (unique, P-XXXXXXXX via generate_patient_code() — docstring: hides DB ids/names from list endpoints), created_at/updated_at. Indexes on patient_code, created_by. Explicit docstring: demographic fields are never fed to the ML model.

patients.PatientAssignment — healthcare_worker FK (limit_choices_to role HCW), patient FK, assigned_by FK (limit ADMIN, SET_NULL), created_at; unique_together(worker, patient).

assessments.Assessment — patient FK (CASCADE, assessments), created_by FK (SET_NULL), visit_date, gestational_week (nullable), the 8 model feature FloatFields (age, body_temperature, heart_rate, systolic_bp, diastolic_bp, bmi, hba1c, fasting_glucose — comment: names match mlcore.constants.FEATURES), symptoms JSONField (fixed vocabulary, rule-engine input), notes, created_at. Ordering [visit_date, id]; index (patient, visit_date). Helper feature_values() returns {f: getattr(self,f) for f in FEATURES}.

> assessments.Prediction — assessment OneToOne (related prediction), risk_level (choices = the 3 levels), probability, probabilities JSON, explanation JSON (SHAP), model_name, model_version, model_ref FK->mlcore.ModelVersion (SET_NULL — prediction stays valid even if the model row is deleted), created_at.

assessments.Alert — assessment FK, patient FK, category (NORMAL/ATTENTION/HIGH_CONCERN/EMERGENCY), rule_id, rule_description, message, rules_version (default v1), status (OPEN/ACKNOWLEDGED/RESOLVED, default OPEN), acknowledged_by FK (SET_NULL), created_at. Indexes (patient,status), (category).

mlcore.ModelVersion — name, version (unique_together), trained_at, dataset_version (md5), features JSON, training_config JSON, metrics JSON, validation_metrics JSON, artifact_path, selection_criterion, status (CANDIDATE/PRODUCTION/RETIRED). Index on status.

kb.KnowledgeDocument — title, source_url, file (upload_to kb/documents/), content_text, sha256, chunk_count, indexed_at, uploaded_by FK, timestamps.

kb.DocumentChunk — document FK (CASCADE, chunks), chunk_index, section, text, embedding JSONField (vector lives here in the local backend), embedding_backend. Index (document, chunk_index).

> chat.ChatSession — title, patient FK (CASCADE, chat_sessions), created_by FK, timestamps. chat.ChatMessage — session FK (messages), role (USER/ASSISTANT), content, escalation JSON, sources JSON, generation_mode (llm | rag_only_fallback | escalation | no_llm_configured), created_at.

reports.Report — patient FK, assessment FK, generated_by FK (SET_NULL), file FileField (reports/), created_at.

audit.AuditLog — user FK (SET_NULL), action (19 choices), target_type, target_id, detail JSON ("Never store medical values or chat content here"), ip_address, created_at (indexed). Index (action, created_at).

Relationship diagram (actual)

```
accounts.User (role: HEALTHCARE_WORKER | ADMIN)
   | created_by / acknowledged_by / assigned_by / generated_by / uploaded_by / created_by ...
   v
patients.PatientProfile <-- patients.PatientAssignment (worker <-> patient, admin-issued)
   | 1:N                        (unique per worker+patient)
   |--> assessments.Assessment --1:1--> assessments.Prediction --FK--> mlcore.ModelVersion
   |            | 1:N
   |            |--> assessments.Alert
   |            `--> reports.Report
   |--> chat.ChatSession --1:N--> chat.ChatMessage
   |--> kb: (no patient link; KnowledgeDocument 1:N DocumentChunk, both admin-scoped)
   `--> audit.AuditLog (polymorphic-by-string target_type/target_id)
```

Why designed this way: one Assessment row = one immutable visit (raw inputs + who entered it); the derived Prediction is separate so model outputs are pinned to the exact model version (model_ref) for reproducibility; alerts are separate rows so they have their own lifecycle (acknowledge/resolve); reports are stored files so downloads are authorization-checked rather than regenerated; audit uses string targets so it can reference anything without FK coupling.

9. Authentication and Authorization

1. Login.jsx form -> authService.login() -> POST /api/auth/login/ with {username,password}.
2. accounts/views.LoginView (subclass of SimpleJWT TokenObtainPairView) -> CustomTokenObtainPairSerializer.validate:

- calls SimpleJWT to authenticate,
- rejects any role outside {HEALTHCARE_WORKER, ADMIN} (legacy PATIENT accounts) with AuthenticationFailed and logs LOGIN_FAILED with reason: role_not_allowed,
- any failure logs LOGIN_FAILED (audit) and re-raises,

- on success logs LOGIN and embeds the full UserSerializer payload as data["user"].

1. Response: {access, refresh, user{...}}. Frontend stores access+refresh in localStorage and sets the context user; navigates to /app/dashboard (or location.state.from).
2. Every subsequent request carries Authorization: Bearer <access> (axios interceptor).
3. On 401, interceptor refreshes once via POST /api/auth/refresh/ (TokenRefreshView -> CustomTokenRefreshSerializer, which re-checks role + is_active from the DB before minting a new access token — closing the "legacy PATIENT refresh loophole" per its docstring).
4. Logout: POST /api/auth/logout/ -> RefreshToken(refresh).blacklist() (requires rest_framework_simplejwt.token_blacklist in INSTALLED_APPS); frontend clears localStorage regardless.
5. Session restore on page reload: AuthContext calls GET /api/auth/profile/ if a token exists; 401 clears tokens.

Actual login flow (traceable)

- Global default: IsAuthenticated (settings). Views then layer IsHealthcareWorker / IsAdmin / object-level checks.
- Object-level access is centralized in patients/selectors.py:
- patient_ids_for_healthcare_worker(user) = created union assigned patient ids,
- can_access_patient(user, profile) — ADMIN: all; HCW: creator-of or assigned-to; otherwise False,
- scoped_patient_queryset(user) — queryset-level enforcement.
- ProtectedRoute.jsx duplicates role checks client-side (UX), but the backend is the authority (verified by tests test_worker_cannot_assess_another_workers_patient, test_admin_endpoints_forbidden_for_workers, etc.).

Token/permission layers This same helper is reused by assessments, alerts, reports, chat, kb-adjacent flows — one definition of "who can touch which patient".

Role behavior matrix (actual)

Capability	HEALTHCARE_WORKER	ADMIN
Login	yes	yes
Create/list patients	own + assigned	all
Assessments/alerts/reports/chat for accessible patients	yes	yes
Acknowledge/resolve alerts	yes (IsHealthcareWorker on AlertAckView)	yes
Upload/delete/reindex knowledge docs	no (403, IsAdminOrReadOnly write side)	yes
Model activate (/api/models/<id>/activate/)	no (IsAdmin)	yes
Audit logs (/api/audit/)	no 403	yes
User management (/api/auth/users/)	no 403	yes
System stats (/api/admin/stats/)	no (IsAdmin)	yes

- accounts/migrations/0002_alter_user_role.py: shrinks role choices to HCW/ADMIN and runs deactivate_legacy_patient_accounts() — sets is_active=False on all role="PATIENT" rows (never deletes them, preserving FKs/audit).
- patients/migrations/0002_remove_patientprofile_user_and_more.py: data migration copies user.full_name/email/phone onto the standalone PatientProfile, then removes the user OneToOneField; adds created_by, contact fields, is_active, archived_at, index.
- accounts/serializers.py guards both login and refresh; accounts/tests.py tests test_legacy_patient_role_cannot_log_in, test_legacy_patient_refresh_rejected, test_legacy_patient_access_token_rejected; patients/tests.py test_system_stats_have_no_patient_role; frontend ProtectedRoute.jsx hard-codes the two valid roles.
- smoke_test.py asserts self-registration (/api/auth/register/) returns 404.

Patient-role removal (already implemented in the working tree)

10. Patient Management

```
Lifecycle -> implementation map:
```

Step	Frontend	API	Backend
Create	pages/hcw/PatientNew.jsx (17-field form, client-required full_name, converts dates/numbers, maps error.details to field errors)	POST /api/patients/	PatientListView.create -> serializer (blood-group normalization, gravidity <=30, gestational week 1-45) -> save(created_by=request.user) -> audit PATIENT_CREATED
List/search	pages/hcw/Patients.jsx (client-side search + risk-first sort)	GET /api/patients/?search=	scoped_patient_queryset + Q filter on code/name/phone/email; patient_summaries batch-aggregates current risk/confidence/trend/counts/open alerts
Detail/update	PatientDetail.jsx + components/patient/ProfilePanel.jsx	GET/PATCH /api/patients/<id>/	PatientDetailView — can_access_patient else ApiError 403; summary attached; audit PATIENT_UPDATED
Archive	(UI shows ARCHIVED chip; archive toggle itself via admin — not exposed as a worker button)	PATCH is_active	field-level update
Assignment	pages/admin/Users.jsx assignment form	POST /api/patients/assignments/, GET, DELETE /api/patients/assignments/<id>/	Admin-only (IsAdmin); audit PATIENT_ASSIGNED
Stats	pages/admin/Dashboard.jsx	GET /api/admin/stats/	SystemStatsView — counts users by role, patients, assessments, predictions, risk distribution, open alerts, chat sessions, KB docs, reports, model info

11. Maternal Risk Assessment

The 8 model features (exactly mlcore/constants.FEATURES, order matters):

Feature	Meaning	UI field (AssessmentPanel.jsx FIELDS)	Validation
age	Age in years	number, 14-55	serializer range (14,55)
body_temperature	Body temperature F	95-106	range (95,106)
heart_rate	Resting HR bpm	40-200	range (40,200)
systolic_bp	Systolic BP mm Hg	70-250	range (70,250); must >= diastolic
diastolic_bp	Diastolic BP mm Hg	40-150	range (40,150)
bmi	Body mass index	12-60	range (12,60)
hba1c	Blood glucose - HbA1c column (dataset units) - observed 30-50	20-60	range (20,60)
fasting_glucose	Blood glucose - fasting column (dataset units) - observed 3.5-8.9 (mmol/L-like)	2-20	range (2,20)

constants.py openly documents the unit inconsistency: the dataset's glucose headers don't match value magnitudes; values are used as-is, recorded rather than silently fixed.

Additionally collected but not model inputs: visit_date (not future), gestational_week (1-45, optional), symptoms (12-token vocabulary from rules.json), notes. Patient demographics (name, DOB, blood group...) are recordkeeping only — PatientProfile docstring says so explicitly.

Serialization path: AssessmentSerializer (ModelSerializer) declares all fields, read_only for id/patient/created_at/prediction, get_prediction embeds the linked prediction; validators: validate_visit_date (no future), validate_gestational_week, validate_symptoms (must be subset of rules.json vocabulary), and a cross-field validate() applying the same PLAUSIBILITY_RANGES used in training-data cleaning plus diastolic <= systolic.

12. Machine Learning Pipeline

```
Files: mlcore/constants.py -> dataset.py -> preprocessing.py -> training.py -> evaluation.py -> registry.py -> explain.py; orchestration
management/commands/train_models.py / inspect_dataset.py; model DB models.py.
```

- backend/data/Mathernal_Risk.csv (duplicate copy at repo root), md5 6ad779f622f39cc28646d92a01270dab, 6,103 rows x 11 columns, no missing values.
- Target Status -> risk_level: low 2001 / mid 2043 / high 2059 (near-balanced 33/33/33 -> resampling deliberately rejected; recorded in TRAIN_CONFIG).
- Columns mapped by COLUMN_MAP (e.g., "Body Temperature(F)" -> body_temperature, "Blood Glucose(HbA1c)" -> hba1c).
- Unused: Patient ID, Name (identifier + PII); LEAKAGE_FEATURES = [] with a documented review (all features are point-of-assessment measurements).
- Cleaning (clean_dataset): drops exact duplicates on features+label (1 row) and implausible rows outside PLAUSIBILITY_RANGES (45 rows: age 1, body_temperature 43, diastolic 1) -> 6,057 rows. Every drop is counted in stats and persisted by inspect_dataset into backend/data/dataset_report.json.

Dataset

- Stratified 70/15/15 (test 0.15; validation = 0.15/0.85 of the remainder), random_state=42 (twice, reproducibly).
- Preprocessor (preprocessing.py): median imputation only, fitted on the training split, serialized into the joblib bundle as a plain dict (to_dict/from_dict) "so the persisted artifact is self-describing". No scaling — docstring: tree ensembles need none; imputer is future-proofing for NaNs.

Split & preprocessing

Models (training.py) RandomForestClassifier(n_estimators=400, min_samples_leaf=2, class_weight="balanced_subsample", random_state=42, n_jobs=-1) XGBClassifier(n_estimators=400, learning_rate=0.08, max_depth=6, subsample=0.9, colsample_bytree=0.8, tree_method="hist", eval_metric="mlogloss", random_state=42)

- Labels encoded to indices of RISK_LEVELS so probability columns always align with the canonical class order for both models.
- quick=True mode trains tiny 10-tree models for tests/smoke runs — never registered as production.
- Persisted bundle per model: {model_name, model, preprocessor(dict), features, classes, dataset_md5, trained_at, train_config} -> ml_artifacts/{randomforest,xgboost}_{full,quick}.joblib.
- evaluate_classifier: accuracy, macro precision/recall/F1, high_risk_recall ("a missed high-risk maternal case is the costliest error"), per-class metrics, confusion matrix, ROC-AUC OVR macro (with probability-column reordering to sklearn's sorted-label expectation).
- selection_score: criterion-driven — default high_risk_recall_then_f1 (tuple: high-risk recall, macro F1, AUC); macro_f1 and accuracy alternatives exist; comment: "Accuracy is never the sole basis."

Evaluation (evaluation.py)

Registration (train_models.py command)

```
Computes winner by criterion, stamps version v1.<YYYYMMDDHHMM>, retires all other rows of that family and (winner case) all other PRODUCTION rows -> single
global production model, inside transaction.atomic(); logs MODEL_TRAINED audit events per row.
```

- get_production_bundle(): thread-safe (threading.Lock) cached load; finds ModelVersion(status=PRODUCTION) newest-first; resolves artifact path relative to BASE_DIR; raises ModelUnavailableError (-> HTTP 503 MODEL_UNAVAILABLE via mlcore/exceptions.py) if none/missing/corrupt — "a prediction is never fabricated".
- predict(features_dict): validates all 8 features present -> builds (1,8) array in FEATURES order -> Preprocessor.from_dict(bundle).transform -> model.predict_proba -> argmax -> risk label; stashes _predicted_index/_predicted_label on the bundle for the SHAP layer; returns {risk_level, probabilities{cls: 4dp}, probability, model{name,version,id}, explanation}.
- feature_importance(bundle): global feature_importances_ sorted desc (serves /api/models/feature-importance/).

Serving (registry.py)

13. Preprocessing

- Training time: Preprocessor().fit(X_train) -> np.nanmedian per column, NaN medians fallback to 0 -> transforms train/val/test.
- Inference time: the medians ride inside the joblib bundle; predict() rebuilds a Preprocessor.from_dict(...) and transforms the single input vector. Training and inference share the same numbers by construction — the class docstring: "Fitted ONLY on training data and persisted together with the model, so inference-time preprocessing is bit-for-bit identical to training."
- Encoding: none needed (all features numeric); label encoding is the fixed index mapping RISK_LEVEL_ORDER.
- Scaling: none (trees don't need it; documented).
- Range checking happens twice: at data-cleaning (drop implausible training rows) and at the API boundary (AssessmentSerializer.validate) — same constants file for both, so the model never sees values it wasn't trained to see.

Exactly one learned transformation exists: median imputation (mlcore/preprocessing.py).

14. Risk Classification

Classification comes from the model's argmax over the three class probabilities — no thresholds, no label-encoder file, no UI-only labels:

registry.predict(): proba = model.predict_proba(X)[0] best = int(np.argmax(proba)) predicted = classes[best] # classes = ["low risk","mid risk","high risk"] from the bundle

- risk_level = argmax class name; probability = its probability; probabilities = full per-class dict.
- The rule engine runs in parallel and does not override the ML class — it produces a separate rules.category and, when != NORMAL, an Alert. The only place rules "outrank" the model is in the chat/UX escalation path (see §16/§21): an emergency rule answer bypasses the LLM and is displayed as the dominant warning; the ML result is still stored and shown.
- The three levels are the dataset's canonical labels (constants.py: "Do not invent others"), mirrored in Prediction.risk_level choices, UI badges, and chart colors.

15. SHAP Explainability

What/why: SHAP (SHapley Additive exPlanations) attributes each feature's contribution to a prediction. In this project it exists so the healthcare worker can see why the model said "high risk" — the explain.py docstring states the wording rule: SHAP values "describe which features contributed most to the MODEL'S PREDICTION. They are never medical causation statements about the patient."

- Where: mlcore/explain.py:explain_prediction(bundle, vector) — called from registry.predict() immediately after probability computation, using the same preprocessed vector.
- How: shap.TreeExplainer(model).shap_values(vector); then a compatibility ladder for sklearn/xgboost output shapes (3-D (1, n_features, n_classes) -> slice predicted class; 2-D; legacy list-per-class). Takes top 8 by absolute impact; each item = {feature, label (human name from FEATURE_LABELS), value, impact (4dp), direction increasing/decreasing/neutral}. Response includes method: "SHAP (TreeExplainer)", target_class, and the disclaimer "statistical contributions, not medical causes."
- Failure behavior: the whole thing is wrapped in try/except — "Explainability must never take the prediction endpoint down" — falls back to {method:"unavailable", features:[]} and still returns the prediction.
- Persistence: stored on Prediction.explanation JSON -> re-shown in Overview/history without recomputation.
- Frontend: charts/index.jsx ShapBars — horizontal Recharts bar chart, red bars (#c0392b) for positive impact (pushed risk up), green (#2e7d5b) for negative, zero reference line; rendered in PredictionResult (right after submission) and in the patient Overview tab, always followed by a Disclaimer.

16. Rule-Based Safety Engine

> Files: assessments/rules/rules.json (data), engine.py (evaluator), keywords.py (chat text -> symptom tokens), __init__.py (exports evaluate, load_rules).

> Categories (ranked in CATEGORY_RANK): NORMAL(0) -> ATTENTION(1) -> HIGH_CONCERN(2) -> EMERGENCY(3).

- EMERGENCY: EMER-CONVULSIONS, EMER-BLEEDING, EMER-BREATHING (symptom match); EMER-BP-HIGH sys >=160; EMER-BP-DIA dia >=110; EMER-TEMP temp >=104F; EMER-HR-HIGH HR >=140.
- HIGH_CONCERN: symptoms severe_headache, blurred_vision, severe_abdominal_pain, decreased_fetal_movement, swelling_face_hands; HC-BP-SYS >=140, HC-BP-DIA >=90, HC-HR-HIGH >=120, HC-FEVER >=100.4F.
- ATTENTION: symptoms persistent_vomiting, dizziness_fainting, fever; ATT-HR-LOW HR <55; ATT-BMI-LOW <18.5; ATT-BMI-HIGH >=30.

The 21 rules (v1) — verbatim thresholds:

rules.json carries a governance note: thresholds are configurable defaults aligned with widely used clinical guidance ranges and "MUST be reviewed and approved by qualified clinical stakeholders before any real-world deployment."

- load_rules() caches JSON keyed by file mtime (hot-reload on edit).
- _matches: symptom rules = token membership; vital rules = operator table _OPS {>=,>,<=,<,==} over the numeric feature; unknown symptom tokens ignored.
- evaluate(vitals, symptoms): collects all triggered rules; worst = highest-rank category; escalate = worst in (EMERGENCY, HIGH_CONCERN); returns category, escalate flag, rules_version, human message from messages block, and the ordered trigger list {id, category, description}.
- Design properties stated in the docstring: explicit (every rule is a JSON row clinicians can inspect), versioned (stamped on every outcome), testable (pure function, no I/O), explainable (ids + descriptions returned).

Mechanics (engine.py):

Precedence with ML: For assessments, ML and rules are independent outputs; both are returned and both persist (Prediction + optional Alert). For chat, chat/services.respond() checks rules first; if escalate, the LLM is bypassed entirely and a deterministic emergency answer is returned (generation_mode: "escalation"), with RULE_ESCALATION audited. The engine docstring: "Emergency rules outrank the ML model, RAG and the

LLM. Nothing downstream may soften or hide an emergency outcome."

Why separate from ML: statistical models can miss deterministic danger thresholds; a rule set is inspectable, auditable, versioned, and guaranteed to fire — a classical safe-ML pattern.

17. Alerts

Creation (assessments path, assessments/services.create_assessment): after prediction, rules_evaluate(feature_values, symptoms) runs; if category != "NORMAL", one Alert row is created with rule_id = up to 5 triggered ids comma-joined, rule_description = their descriptions, message = category message, rules_version. Stored inside the same transaction.atomic() as the assessment/prediction (alert and assessment succeed or fail together).

> `Lifecycle: status OPEN -> ACKNOWLEDGED -> RESOLVED via POST /api/assessments/alerts/<pk>/ack/ (AlertAckView, IsHealthcareWorker, AlertAckSerializer restricts to those two statuses; records acknowledged_by; access-checked through can_access_patient).`

Retrieval: GET /api/assessments/alerts/?patient=&status=&category= (AlertListView) — HCWs automatically scoped to their patients; ADMIN sees all.

- pages/hcw/Alerts.jsx — global open-alerts table (fetches status:OPEN), category filter dropdown, severity sort (rank map), Ack/Resolve buttons (row disappears on ack).
- components/patient/AlertsPanel.jsx — per-patient list, all statuses, same ack/resolve, optimistic status update.
- Dashboard cards count open/emergency alerts; PatientDetail Overview shows an emergency banner and an "Active alerts" strip.
- Styling maps categories to risk-low/mid/high/emergency classes; color is always paired with an icon + text label (UI and CSS comments both state this accessibility rule).

Display:

18. Risk History

- Storage: every assessment persists raw inputs (Assessment) and model output (Prediction) with timestamps and model identity — history is a natural consequence.
- Retrieval: GET /api/assessments/risk-history/?patient=<id> -> RiskHistoryView -> services.prediction_series(patient): predictions joined to assessments, ordered by visit_date, id (oldest first), each {id, risk_level, probability, model "Name version", visit_date, gestational_week} + the standard disclaimer.
- UI: HistoryPanel (tab "History & Trend") shows the sequence line ("low risk -> mid risk ..."), the RiskTrendChart step line chart, and a full table (visit date, gestational week, risk badge, confidence %, model, per-row "PDF" button that generates + downloads a report).
- patient_summaries (patients/selectors.py) computes per-patient assessment_count, last_assessment_date and the latest stored prediction for list views.

19. Risk Trend

- Empty -> {status:"no_data", direction:"unknown"}.
- current = last risk level; previous = second-to-last if >=2 predictions.
- One visit -> status:"baseline".
- Otherwise diff = RISK_LEVEL_ORDER[current] - RISK_LEVEL_ORDER[previous] (low=0, mid=1, high=2):
- direction: "increasing" (diff>0) / "decreasing" (diff<0) / "stable";
- status: increasing -> "increasing", decreasing -> "improving", stable -> "stable" ("improving means a move to a lower-risk class").
- Returns also sequence (all levels) and visits count.

Algorithm (assessments/trends.compute_trend, input = prediction series oldest-first):

- trends.py docstring: "This is descriptive analytics over stored predictions — the system does NOT predict future risk (no temporal model is trained; the dataset has no visit-level fields)."
- RiskTrendView response carries the literal disclaimer "Trends describe stored assessments only. The system does not predict future risk."; tests assert that string.
- UI repeats it under every chart (PatientDetail overview, HistoryPanel), and the PDF says the same.
- The only temporal ML code is the explicitly quarantined research module mlcore/synthetic.py (Cholesky-sampled synthetic visit sequences, always flagged is_synthetic, quality-gated, "NEVER mixed into production training").

Historical trend != future prediction — enforced in code and copy:

20. PDF Reports

> `Flow: Assessment -> POST /api/reports/ {"assessment": id} (ReportListCreateView.post) -> access check (can_access_patient), requires an existing prediction -> reports/services.generate_report(assessment, generated_by):`

- collects prediction, patient, full prediction_series + compute_trend;
- builds an A4 ReportLab document (SimpleDocTemplate into BytesIO): title + generated-at, color-coded risk category (per-level RGB constants), confidence %, per-class probabilities, model name/version, patient identifier (patient_code — not the name), assessment date, gestational week, an 8-row measurements table, risk trend line + direction with the "does not predict future risk" note, top-6 SHAP bullets ("increased/decreased the model's risk estimate (impact +/-x.xxx)") + SHAP disclaimer, rule-based alerts (or "No warning rules triggered"), and the full DISCLAIMER.
- saves via Report.file.save(filename, ContentFile(...)) -> media/reports/report_<code>_<assessmentId>_<timestamp>.pdf; returns the Report row.

```
Response includes download_url; download is GET /api/reports/<pk>/download/ -> ReportDownloadView -> can_access_patient check ->
FileResponse(as_attachment=True).
```

```
Why backend-side PDF generation: single source of truth (prediction, SHAP, trend, alerts all already live server-side), consistent formatting, and the download
endpoint enforces authorization — the frontend just streams the blob (utils/reportDownload.js: getRaw -> Blob -> object URL -> programmatic <a download> click
-> revoke). Note: pages/hcw/Reports.jsx also renders a direct /media/… link (see §38/§43 — it bypasses the auth-checked endpoint and depends on DEBUG media
serving).
```

21. AI Assistant

Pipeline (chat/services.respond, docstring diagram matches the code exactly):

```
message -> emergency rule engine
               |-- triggered -> escalation answer (LLM bypassed entirely)
               `-- clear -----> patient context -> RAG retrieval -> LLM
                                                       `-- unavailable -> safe fallback
```

Step-by-step:

1. User ChatMessage persisted immediately.

1. detect_symptoms(text) (rules/keywords.py): lowercases, matches phrase lists (e.g., "convulsion/seizure/fit/eclampsia" -> convulsions; "baby not moving" -> decreased_fetal_movement) — purely lexical, no LLM.
2. Rules first: if any symptoms detected, the latest assessment's vitals (sys/dia BP, HR, temp, BMI) are loaded and rules_evaluate(vitals, detected) runs; a comment explains the design: only high-concern/emergency words in the message itself escalate — stored vitals alone never hijack a chat about nutrition (vitals are only injected when symptoms were detected).
3. Escalation branch: deterministic answer built from rules_out['message'] + triggered rule ids + "This chat assistant cannot provide emergency care."; saved with generation_mode="escalation"; audited RULE_ESCALATION; returned without touching RAG or the LLM.
4. Normal branch: _patient_context() fetches the latest prediction only -> one line "latest assessed risk category: ... (model confidence ...%)" (+ gestational week) — docstring: "Minimal, authorized context (no sensitive history)".
5. rag.retrieve_context(question) -> top chunks (§22).
6. History: last 12 messages of the session (the final user turn is dropped and passed explicitly as question).
7. llm.generate_answer(question, context_chunks, history, patient_context) -> (answer, mode).
8. Fallbacks: LLM unconfigured/fails/empty -> if RAG found chunks, answer = FALLBACK_NO_LLM + verbatim excerpts with [title - chunk n] headers (mode="rag_only_fallback"); else FALLBACK_NO_LLM + "No indexed knowledge matched…" (mode="no_llm_configured").
9. Assistant message persisted with sources (title, url, chunk_index, score), escalation summary, generation_mode; audited CHAT_ACCESS; serialized with CHAT_DISCLAIMER.

Chat storage: ChatSession per patient (title auto-set from first message's first 60 chars), ChatMessage rows per turn.

LLM call (chat/llm.py): OpenAI-compatible client (api_key=settings.LLM_API_KEY, base_url, timeout=LLM_TIMEOUT_SECONDS), model settings.LLM_MODEL (default gpt-4o-mini), temperature=0.3, max_tokens=600. System prompt forbids diagnosing, prescribing, certainty claims, and claiming to have examined the patient; demands grounding in provided excerpts, cautious language, <250 words. generate_answer raises nothing — any exception is logged and the fallback tuple is returned.

The assistant does NOT diagnose — enforced by the prompt, disclaimers in every reply payload, and the chat UI's Disclaimer line.

22. RAG / Knowledge Base

Stage	File	Detail
Ingestion	kb/views.py KnowledgeDocumentListCreateView.perform_create	Admin uploads PDF/TXT/MD or pastes text; requires one of them; sha256 of content stored; on indexing ValueError the document is deleted and a 400 returned
Extraction	kb/services/ingestion.extract_text	.pdf via pypdf (PdfReader, page join), .txt/.md/.text read as UTF-8, else content_text
Chunking	ingestion.chunk_text	fixed 900-char chunks with 150-char overlap (constants CHUNK_SIZE, CHUNK_OVERLAP); empty text -> []
Embedding	kb/services/embeddings.py	local: TfidfVectorizer(max_features=5000, stop_words="english") fitted over the whole corpus (fit_corpus called from indexing with all chunks of all documents; other documents' chunks are re-embedded after refit via _reembed_others); openai: text-embedding-3-small
Indexing	kb/services/indexing.index_document	transaction.atomic; deletes old chunks, embeds, bulk_creates DocumentChunk rows with vector lists + backend tag; sets chunk_count, indexed_at
Storage	kb/models.DocumentChunk.embedding JSONField	the vector store is the relational DB in the default backend
Retrieval	kb/services/store.retrieve + rag.retrieve_context	loads all chunk vectors (SQL), L2-normalizes, cosine = matrix @ query-transpose, top-k; MIN_SCORE = 0.05 filter (below -> "too weak to cite"); returns {document_id, document_title, source_url, chunk_index, text (<=1200 chars), score}; never raises — empty/failed KB -> None
Citation	chat/services -> ChatMessage.sources -> ChatPanel collapsible "Knowledge-base sources (n)" list with title, chunk index, score, and external link	every non-escalation answer carries its sources
Admin APIs	GET/POST /api/rag/documents/, GET/PATCH/DELETE .../<pk>/, GET .../chunks/, POST /api/rag/reindex/, POST /api/rag/retrieve/	upload uses MultiPart+Form+JSON parsers; writes are admin-only (IsAdminOrReadOnly); reindex admin-only; retrieve is authenticated for probing (Knowledge page "Retrieval probe")

23. Vector Database / Qdrant

- Default backend is NOT Qdrant — it is the local numpy/SQLite cosine scanner described above (settings.VECTOR_BACKEND default "local"; qdrant_client is not installed in the venv — verified).
- kb/services/store.py implements the abstraction: retrieve() dispatches to _retrieve_qdrant only when VECTOR_BACKEND == "qdrant" and VECTOR_DB_URL is set; if the client package is missing it logs a warning and falls back to local retrieval. Qdrant path: QdrantClient(url, api_key) -> client.search(collection_name="aimhra_kb", query_vector=..., limit=top_k) -> [(hit.id, hit.score)].
- What vectors represent: TF-IDF (5000-dim, corpus-fitted) or OpenAI 1536-dim embeddings of 900-char document chunks; the query vector embeds the user's question with the same backend — a prerequisite for meaningful cosine similarity.
- Settings involved: VECTOR_BACKEND, VECTOR_DB_URL, VECTOR_DB_API_KEY (env; example file has them empty).

24. API Reference (all endpoints, from config/urls.py + app urls.py files)

All require JWT unless noted. Success envelope {success:true,data}; errors {success:false,error{code,message,details}}.

Auth — /api/auth/ (accounts/urls.py)

Method & URL	Auth	Body/Params	View -> Logic	Errors
POST /login/	public	{username,password}	LoginView->CustomTokenObtainPairSerializer (role guard, audit LOGIN/LOGIN_FAILED)	401 structured
POST /refresh/	public (refresh token)	{refresh}	TokenRefreshView->CustomTokenRefreshSerializer (re-validates role+active)	401
POST /logout/	any	{refresh}	LogoutView blacklists refresh	400 invalid/missing
GET/PATCH /profile/	any	—	ProfileView (retrieve/update self, audit PROFILE_UPDATED)	401
POST /change-password/	any	{current_password,new_password}	ChangePasswordView; validates current pw + Django validators	400
POST /password-reset/	public	{email}	PasswordResetRequestView — always generic response (no user enumeration); sends reset link (console backend in dev)	always 200
GET/POST /users/	ADMIN	— / {username,email,full_name,phone,role,password}	UserManagementView; validates + hashes password; audit USER_UPDATED	403 non-admin
GET/PATCH/DELETE /users/<pk>/	ADMIN	—	UserManagementDetailView; DELETE = soft (is_active=False)	403

Patients — /api/patients/

Method & URL	Auth	Notes
GET "" ?search=	any staff	scoped list + summaries
POST ""	any staff	create (creator auto-granted)
GET/PATCH <int:pk>/	staff + access	detail/update
GET/POST assignments/	ADMIN	list/create assignments
DELETE assignments/<int:pk>/	ADMIN	remove assignment
GET (prefix /api/admin/) stats/	ADMIN	SystemStatsView dashboard counts

Assessments — /api/assessments/

Method & URL	Auth	Body/Params	Flow	Errors
POST ""	staff + patient param	{patient, visit_date, gestational_week?, 8 features, symptoms[], notes}	serializer validation -> create_assessment (predict->SHAP->rules->atomic persist->trend)	400 validation, 403 access, 503 MODEL_UNAVAILABLE
GET "" ?patient=	staff	optional filter	scoped list w/ pagination	403
GET <int:pk>/	staff + access	—	AssessmentDetailView	404/403
GET risk-history/ ?patient=	staff + access	—	prediction_series + disclaimer	400 missing patient, 403
GET risk-trends/ ?patient=	staff + access	—	trend + series + non-predictive disclaimer	400/403
GET alerts/ ?patient=&status=&category=	staff	—	scoped alert list	403
POST alerts/<int:pk>/ack/	IsHealthcareWorker + access	{status: ACKNOWLEDGED|RESOLVED}	status update + acknowledged_by	400/403/404

Models — /api/models/ (mlcore/urls.py)

Method & URL	Auth	Purpose
GET ""	staff	all ModelVersion rows (full metric payloads)
GET compare/	staff	latest RF vs XGB metrics + production + criterion
POST <int:pk>/activate/	ADMIN	promote to PRODUCTION, retire others, force-reload bundle, audit MODEL_ACTIVATED
GET feature-importance/	staff	global importances of production model (503 if none)
GET diagnostics/	staff	{available, production} or {available:false, reason} (admin dashboard status card)

RAG — /api/rag/ (kb/urls.py) GET/POST documents/ (read: staff; write: admin; multipart or JSON); GET/PATCH/DELETE documents/<pk>/ (re-index on content change); GET documents/<pk>/chunks/; POST reindex/ (admin); POST retrieve/ {question, top_k?} (retrieval-only probe).

Chat — /api/chat/

```
GET/POST sessions/ (scoped by patient access; POST requires patient); GET/DELETE sessions/<id>/ (delete: owner or admin); POST sessions/<id>/send/ {message}
(<=4000 chars; empty -> 400) -> full safety pipeline -> {reply}.
```

Reports — /api/reports/

```
GET "" ?patient= (scoped, latest 50); POST "" {assessment} (201 -> {report, download_url}); GET <int:pk>/download/ (access-checked FileResponse).
```

Audit — /api/audit/ GET "" ?action=&username= (ADMIN only), paginated.

Meta GET /api/schema/, GET /api/docs/ (Swagger UI), /admin/ (Django admin, all models registered with list displays/filters/inlines).

25. Frontend Pages

Page	Route	Who	Purpose / API / behavior
pages/auth/Login.jsx	/login	public	Username+password card; login(); error alert from apiError; disclaimer; note "for Healthcare Workers and Administrators"
pages/hcw/Dashboard.jsx	/app/dashboard (HCW)	HCW/ADMIN	Parallel Promise.all of patients, OPEN alerts, assessments -> StatCards (total/high/mid/low, open/emergency alerts), high-priority alert table, high-risk patient list, recently added, recent assessments; every row links into PatientDetail with ?tab=
pages/hcw/Patients.jsx	/app/patients	HCW/ADMIN	Fetch all once; client-side search (code/name/phone/email) + risk-first sort (high->mid->low, then newest); columns: code, name, contact, computed age from DOB, gest. wk, current risk badge + confidence, trend arrow + status, assessments, last visit, open alerts; Open/Assess buttons
pages/hcw/PatientNew.jsx	/app/patients/new	HCW/ADMIN	17-field form; requires full_name client-side; null/number coercion; on 400 maps error.details onto field errors; navigates to new patient
pages/hcw/PatientDetail.jsx	/app/patients/:id	HCW/ADMIN	The workspace. Loads patient + trend + alerts + assessments in parallel; 7 URL-driven tabs (?tab=): overview, info (ProfilePanel), assessment (AssessmentPanel), history (HistoryPanel), alerts (AlertsPanel), chat (ChatPanel), reports (ReportsPanel). generateReport() = generate + immediate download
pages/hcw/Alerts.jsx	/app/alerts	HCW/ADMIN	OPEN alerts table, category filter, severity sort, Ack/Resolve
pages/hcw/Reports.jsx	/app/reports	HCW/ADMIN	Latest 50 reports list; download via raw /media/ href (see §43)
pages/admin/Dashboard.jsx	/app/dashboard (ADMIN)	ADMIN	adminService.stats() + modelService.diagnostics() -> StatCards, RiskDistributionChart pie, platform activity list
pages/admin/Users.jsx	/app/admin/users	ADMIN	User table + create form (role select HCW/ADMIN), activate/deactivate toggle, patient-assignment form + assignment table
pages/admin/Models.jsx	/app/admin/models	ADMIN	compare + list + feature importance; metric bars, comparison table, Activate button per non-production row, two confusion-matrix heat tables, global importance chart, version history
pages/admin/Knowledge.jsx	/app/admin/knowledge	ADMIN	Upload form (title, source URL, file or pasted text -> FormData), delete with confirm, re-index all, retrieval probe
pages/admin/Audit.jsx	/app/admin/audit	ADMIN	Filterable (action select from 17 constants, username) paginated table with JSON detail column

26. Frontend Components

Component (file)	Props	What it does
Layout.jsx	none (context)	App shell: role-based sidebar nav (NAV_BY_ROLE), user chip, sign-out (logout -> /login), Outlet
ProtectedRoute.jsx	roles?, children	Guard: booting -> Loading; !user -> /login (preserving from); role not in {HCW,ADMIN} -> /login; role not in roles -> own dashboard
ui.jsx - RiskBadge	level,size	Color+icon+text badge (LOW / MID ! / HIGH !!); unknown levels handled
ui.jsx - CategoryBadge	category	Rule categories incl. EMERGENCY
ui.jsx - StatCard	label,value,sub,tone	Dashboard metric card (tones good/warn/danger)
ui.jsx - Loading/ErrorState/EmptyState/Disclaimer	—	Standard async states; ErrorState shows code+message+retry; Disclaimer default text
charts/index.jsx - RiskTrendChart	series	Recharts step-after line over visits (Y = 0/1/2 mapped Low/Mid/High; X = gestational week or Visit n; tooltip shows date + level)
charts - RiskDistributionChart	counts	Donut pie with per-risk colors
charts - ShapBars	features	Horizontal bars, red = positive impact, green = negative, zero line
charts - FeatureImportanceChart	importance	Global model importances
charts - ConfusionMatrix	matrix	Heat-colored HTML table (Actual down / Predicted across)
charts - MetricBars	models	Grouped bars for RF vs XGB across six metrics
patient/AssessmentPanel.jsx	patient,onViewHistory,onAskAssistant	The assessment form (§31) + PredictionResult (emergency/high-concern banners, risk badge, probability bars, SHAP chart, trend-vs-previous, rules applied, disclaimer, next actions)
patient/HistoryPanel.jsx	patientId	Trend + all-assessments table + per-row PDF generate/download
patient/AlertsPanel.jsx	patientId,showStatus	Per-patient alerts with ack/resolve
patient/ProfilePanel.jsx	patient,onSaved	Editable profile form (same fields as PatientNew), success notice, field errors
patient/ChatPanel.jsx	patientId	Session sidebar (new/open/delete), message stream, optimistic user echo, typing dots, escalation badge, collapsible sources, generation-mode line, Enter-to-send, 4000-char cap
patient/ReportsPanel.jsx	patientId	Reports table via authorized download only
constants/clinical.js	—	BLOOD_GROUPS + bloodGroupOptions() — keeps unknown legacy stored values

		selectable so an edit never blanks them; comment says keep in sync with the backend validator
utils/reportDownload.js	—	Blob -> object URL -> temporary anchor click -> revoke

27. Frontend Services

- authService: login (stores tokens, returns user), logout (blacklist + always clears tokens), profile, updateProfile, changePassword, requestPasswordReset.
- patientService: list(params), create, detail(id), update(id,payload), assignments(), createAssignment, deleteAssignment(id).
- assessmentService: create(payload), list(params), history(patientId), trend(patientId), alerts(params), ackAlert(id,status).
- modelService: list, compare, activate(id), featureImportance, diagnostics.
- chatService: sessions(patientId,params), createSession(patientId,title), session(id), send(id,message), deleteSession(id).
- kbService: documents, upload(FormData), deleteDocument, reindex, retrieve(question), chunks(id).
- reportService: list(params), generate(assessmentId), download(id) (blob).
- adminService: stats, users, createUser, updateUser, auditLogs(params).

All in frontend/src/services/auth.js (single "domain services" module; comment: "every backend endpoint wrapped in one place"):

Each maps 1:1 to an endpoint (URL, method, auth via interceptor, envelope unwrapping). Why this layer exists: components never construct URLs or handle tokens; changing an endpoint touches exactly one function; error normalization is centralized.

28. Configuration and Environment Variables

backend/.env.example (values read in config/settings.py):

Variable	Read at	Meaning	Secret?
SECRET_KEY	settings.17	Django signing key	YES - must be changed in prod
DEBUG	settings.18	dev mode (adds testserver host; verbose errors via middleware)	—
ALLOWED_HOSTS	settings.19	comma list	—
DATABASE_URL	settings.82	optional postgres/mysql switch	YES if it embeds credentials
CORS_ALLOWED_ORIGINS	settings.144	allowed frontend origins	—
JWT_ACCESS_MINUTES / JWT_REFRESH_DAYS	settings.137-138	token lifetimes (30 min / 7 days)	—
DATASET_PATH	settings.165	training CSV location	—
MODEL_SELECTION_CRITERION	settings.167	model promotion criterion	—
ML_ARTIFACTS_DIR	settings.166	joblib artifacts dir	—
LLM_API_KEY	settings.172	LLM auth - empty => whole system runs LLM-free	YES
LLM_BASE_URL / LLM_MODEL / LLM_TIMEOUT_SECONDS	settings.173-175	OpenAI-compatible endpoint/model/timeout	—
EMBEDDING_BACKEND	settings.176	local (TF-IDF) or openai	—
VECTOR_BACKEND / VECTOR_DB_URL / VECTOR_DB_API_KEY	settings.177-179	vector store selection + Qdrant creds	API key = YES
EMAIL_BACKEND / DEFAULT_FROM_EMAIL	settings.184-185	password-reset delivery (console in dev)	—

frontend/.env.example: VITE_API_BASE_URL=/api — dev works through the Vite proxy; set the full origin in production. vite.config.js also honors VITE_API_PROXY_TARGET.

Critical finding: backend/.env.example line 22 contains what appears to be a real OpenAI API key committed in plaintext. It is deliberately not reproduced here. It should be treated as compromised: revoke/rotate the key and scrub it from git history. (The file is also listed as modified in git status.)

29. Package Dependencies

- backend/requirements.txt (15 entries, version ranges) — each one's role is in §3. Two problems found: requests is unused; pypdf is missing though imported by kb/services/ingestion.py (installed ad-hoc in the venv at 6.16.1, so ingestion works here but not on a clean install); dj_database_url referenced only for Postgres (not installed).
- frontend/package.json — scripts: dev (vite), build, lint (oxlint), preview. Runtime deps: axios, react, react-dom, react-router-dom, recharts. Dev: types, plugin-react, oxlint, vite. package-lock.json exists (lockfile present in the repo listing; not read due to size).
- No Python lockfile / no Pipfile/pyproject.toml — pinned only by loose ranges.

30. Database Migrations

Migration	What it does / why
accounts/0001_initial	Creates custom User. Note: header says "Django 6.1" and the original role choices were PATIENT/HEALTHCARE_WORKER/ADMIN with default PATIENT - fossil evidence of the pre-refactor design
accounts/0002_alter_user_role	Role choices -> HCW/ADMIN (default HCW) + data migration deactivate_legacy_patient_accounts (sets is_active=False for PATIENT rows; reverse = noop)

patients/0001_initial	PatientProfile with a user OneToOne + PatientAssignment; indexes/unique constraints
patients/0002_remove_patientprofile_user_and_more	The standalone-record refactor: adds contact/notes/is_active/archived_at/created_by fields, data migration copy_legacy_user_data_to_patient_record (copies name/email/phone from the linked user), removes the user FK, adds created_by index
assessments/0001_initial	Assessment/Prediction/Alert tables + indexes; Prediction FK -> mlcore.ModelVersion
mlcore/0001_initial	ModelVersion with unique (name,version), status index
kb/0001_initial	KnowledgeDocument + DocumentChunk (embedding JSON)
chat/0001_initial	ChatSession + ChatMessage (escalation/sources/generation_mode)
reports/0001_initial	Report (file, patient, assessment, generated_by)
audit/0001_initial	AuditLog (17 original actions)
audit/0002_alter_auditlog_action	Adds PATIENT_CREATED / PATIENT_UPDATED choices to match the new patient endpoints

The 0002 trilogy (accounts/patients/audit, all timestamped 2026-09-06) is one coordinated refactor: patients stop being users; history is preserved, never deleted.

31. Tests

All backend tests are Django TestCase (no pytest config found). Frontend tests: none (only oxlint).

Module	Coverage highlights
accounts/tests.py	Only two roles exist (ROLE_PATIENT absent asserted); default role; register endpoint 404; HCW/Admin login; legacy PATIENT login 401 + refresh 401 + access-token 401; inactive user; failed login -> structured error + audited; refresh round-trip; change password; worker 403 on /api/audit/ and /api/auth/users/; admin user management (no PATIENT in roles); logout blacklists refresh
patients/tests.py	Worker creates patient (code P-..., creator recorded); code generated server-side; creation audited; unauthenticated 401; admin sees all; worker sees only own; search by code/name/phone/email; blood-group validation incl. Rh-less "AB" accepted with readable error; client cannot spoof created_by; system stats exclude PATIENT
assessments/tests.py	RuleEngineTests: ruleset versioned; sys 170 -> EMERGENCY; bleeding symptom -> EMERGENCY; sys 145 + headache -> HIGH_CONCERN; normals stay NORMAL; unknown symptoms ignored; keyword detection. TrendTests: improving/increasing/baseline. AssessmentAPITests (quick-trained real model): create -> 201 with valid risk; emergency vitals create an EMERGENCY Alert; SHAP persisted; range violations 400; dia>sys 400; unknown symptom 400; future date 400; cross-worker 403 with no row created; assigned-patient access; admin any-patient; history/trend endpoints; 503 MODEL_UNAVAILABLE when no model + nothing fabricated
mlcore/tests.py	Dataset loads (6103, 3 classes); cleaning drops documented rows only; identifiers excluded / no leakage features; Preprocessor fit/transform/impute + roundtrip + unfit guard; quick-train once in setUpTestData -> production bundle exists; prediction shape/labels/probability-sum roughly 1; model version recorded; explanation structure + disclaimer; missing feature -> ModelUnavailableError; no model -> 503 shape
chat/tests.py	Worker session creation bound to own patient; cross-patient session 403; cross-worker session open 403; emergency message bypasses LLM (generation_mode=escalation, audited); safe answer without LLM; RAG sources attached when KB populated; no fabricated sources on empty KB; both messages persisted; empty message 400; title from first message; owner-only delete
kb/tests.py	Chunking splits with overlap; empty -> no chunks; indexing persists embeddings + counts; empty doc raises; retrieval returns scored sources with right title; empty KB -> None; unrelated query scores <0.5; admin upload 201; worker upload 403; retrieve-on-empty reports retrieved:false; reindex admin-only
reports/tests.py	Quick-trained model; worker generates report for own patient (starts with %PDF, >1000 bytes); Report row created; download 200 for owner, 403 for stranger; API generation audited
backend/scripts/smoke_test.py	Not a unittest — runnable E2E: worker login, register-404, patient create, emergency assessment, history/trend, chat escalation + safe path, admin KB upload + retrieval, PDF generate/download, model compare (asserts production is XGBoost), audit visibility/403

Missing/weak areas: no frontend tests; no API tests for PasswordResetRequestView email path, ReindexView happy path with documents, ProfileView.update, model activate 404; no security tests (rate limiting, CORS); tests rely on a quick model so metric-quality is untested.

32. End-to-End User Workflows (traced)

Workflow 1 — Login

```
Login.jsx submit -> authService.login -> POST /api/auth/login/ -> CustomTokenObtainPairSerializer.validate (role guard, audit) -> {access,refresh,user} -> localStorage + setUser -> navigate /app/dashboard -> ProtectedRoute passes -> RoleDashboard picks dashboard by role. Failure: wrong password -> 401 envelope -> red alert with backend message; legacy role -> same 401 (audited as role_not_allowed).
```

Workflow 2 — Add patient

```
PatientNew.jsx -> client check full_name -> coerce numbers/dates -> patientService.create -> POST /api/patients/ -> serializer validation (blood group, gravidity, gestational week) -> save(created_by=request.user) -> audit PATIENT_CREATED -> 201 with generated patient_code -> navigate to /app/patients/<id>?tab=overview. Failure: 400 with details -> per-field errors + banner.
```

Workflow 3 — Risk assessment

```
Form -> validate() (all 8 fields required, min/max, dia<=sys, week 1-45) -> POST /api/assessments/ with patient in body -> _resolve_patient (explicit patient + access check) -> serializer (ranges from PLAUSIBILITY_RANGES, symptom vocabulary, no future date) -> create_assessment:
```

- float-coerce the 8 features -> model_predict (registry: load bundle -> preprocessor -> predict_proba -> argmax) ->
- rules_evaluate(vitals, symptoms) ->
- transaction.atomic(): Assessment.create, Prediction.create (risk, probs, SHAP, model name/version/ref), Alert.create if category != NORMAL ->
- audit x2 (+RULE_ESCALATION if escalated) -> prediction_series + compute_trend ->
- response {assessment, risk_level, probability, probabilities, model, explanation, rules, alerts, trend, disclaimer}. UI renders PredictionResult. Failure paths: missing model -> 503 MODEL_UNAVAILABLE (nothing saved); validation -> 400 with field details mapped to inputs; permission -> 403 ErrorState.

Workflow 4 — View history

```
HistoryPanel mount -> assessmentService.trend(patientId) -> GET /api/assessments/risk-trends/?patient= -> _resolve_patient -> prediction_series + compute_trend -> {trend, series, disclaimer} -> RiskTrendChart + sequence line + table (per-row PDF).
```

Workflow 5 — Generate report

```
Button (Overview or History) -> reportService.generate(assessmentId) -> POST /api/reports/ -> access + prediction checks -> generate_report (ReportLab; trend + SHAP + alerts + disclaimer) -> Report.file.save -> 201 {report, download_url} -> downloadReport(id) -> GET /api/reports/<id>/download/ -> FileResponse blob -> browser save.
```

Workflow 6 — AI chat

```
ChatPanel mount -> load sessions (scoped) -> open first / create new -> type -> POST /api/chat/sessions/<id>/send/ -> title auto-set -> respond() pipeline ($21) -> reply persisted -> optimistic UI update -> sources expandable, escalation banner if triggered. Failures: empty/long -> 400; API error -> assistant bubble "Sorry - <message>"; LLM down -> fallback modes shown in UI.
```

33. Feature -> Code Location Map

Feature	Frontend	Backend (view/service)	Database	Main files
Login	pages/auth/Login.jsx, context/AuthContext.jsx	accounts/views.LoginView, accounts/serializers	accounts_user	api/client.js, config/settings.py (SIMPLE_JWT)
JWT/refresh/logout	api/client.js interceptors	TokenRefreshView, LogoutView, custom refresh serializer	token_blacklist tables	accounts/urls.py
Patient management	PatientNew.jsx, Patients.jsx, ProfilePanel.jsx	patients/views.py, selectors.py	patients_patientprofile, patients_patientassignment	patients/serializers.py
Patient assignment	admin/Users.jsx	PatientAssignmentListView/DetailView	same	patients/urls.py, admin_urls.py
Risk assessment	patient/AssessmentPanel.jsx	assessments/views.AssessmentListCreateView, assessments/services.create_assessment	assessments_assessment	assessments/serializers.py
ML prediction	probability bars in PredictionResult	mlcore/registry.predict	assessments_prediction (+ mlcore_modelversion)	mlcore/training.py, ml_artifacts/*.joblib
SHAP explanation	charts.ShapBars	mlcore/explain.explain_prediction	Prediction.explanation JSON	mlcore/constants.FEATURE_LABELS
Rule engine / safety	rule lists in PredictionResult, CategoryBadge	assessments/rules/engine.py + rules.json	rules_version stamped on Alert/ChatMessage	rules/keywords.py (chat)
Alerts	Alerts.jsx, AlertsPanel.jsx, dashboard cards	AlertListView, AlertAckView, alert creation in services	assessments_alert	assessments/urls.py
Risk history	HistoryPanel.jsx	RiskHistoryView, prediction_series	Assessment+Prediction	assessments/services.py
Risk trend	charts.RiskTrendChart, trend cards	RiskTrendView, trends.compute_trend	(derived)	assessments/trends.py
PDF reports	ReportsPanel.jsx, utils/reportDownload.js	reports/views.py, reports/services.generate_report	reports_report (+ media/reports/)	reportlab
AI assistant	patient/ChatPanel.jsx	chat/views.py, chat/services.respond, chat/llm.py	chat_chatsession, chat_chatmessage	rules/keywords.py
RAG / KB	admin/Knowledge.jsx	kb/views.py, kb/services/*	kb_knowledgedocument, kb_documentchunk	kb/urls.py
Vector search	—	kb/services/store.py, rag.py	DocumentChunk.embedding / optional Qdrant	settings.VECTOR_BACKEND
LLM	—	chat/llm.py	(no storage of prompts beyond messages)	.env LLM_*
Authentication guards (FE)	ProtectedRoute.jsx, App.jsx	role checks server-side	—	Layout.jsx
Audit logs	admin/Audit.jsx	audit/views.py, audit/services.log_event	audit_auditlog	called from every app
Model management	admin/Models.jsx, admin/Dashboard.jsx	mlcore/views.py	mlcore_modelversion	train_models command

34. File -> Responsibility Map

File	Responsibility	Key items	Used by
backend/manage.py	CLI entry	execute_from_command_line	dev ops
backend/config/settings.py	All configuration	apps, middleware, DRF/JWT/CORS/DB/ML/LLM/logging	everything
backend/config/urls.py	Route table	10 includes + schema/docs	Django
config/wsgi.py / asgi.py	Server adapters	standard	deployment
core/permissions.py	Role permissions	IsHealthcareWorker, IsAdmin, IsAdminOrReadOnly	accounts, patients, assessments, mlcore, kb, audit
core/exceptions.py	Error envelope	ApiError, structured_exception_handler, flatten_error_messages	DRF globally; raised everywhere
core/middleware.py	JSON for non-DRF errors	StructuredErrorMiddleware	settings
core/pagination.py	Page envelope	StandardPagination	list views

core/responses.py	ok() helper	success envelope	all views
accounts/models.py	User + role	save() derives is_staff	auth everywhere
accounts/serializers.py	Login/refresh guards, user serializing	CustomTokenObtainPairSerializer, CustomTokenRefreshSerializer, ChangePassword, UserAdminSerializer	views + SIMPLE_JWT settings
accounts/views.py	Auth + user mgmt endpoints	LoginView, LogoutView, ProfileView, ChangePasswordView, PasswordResetRequestView, UserManagementView/DetailView	accounts/urls.py
patients/models.py	Patient records	generate_patient_code, PatientProfile, PatientAssignment	assessments/chat/reports
patients/selectors.py	Access-control + summaries	can_access_patient, scoped_patient_queryset, patient_ids_for_healthcare_worker, patient_summaries	patients, assessments, reports, chat views
patients/serializers.py	IO validation	blood-group/gravidity/week validators; summary method fields	views
patients/views.py	Patient + assignment + stats APIs	PatientListView, PatientDetailView, assignment views, SystemStatsView	urls
patients/admin_urls.py	/api/admin/stats/	1 route	config urls
assessments/models.py	Clinical tables	Assessment.feature_values(), Prediction, Alert	services, views, reports
assessments/serializers.py	Input validation	AssessmentSerializer, AlertSerializer, AlertAckSerializer	views
assessments/services.py	The pipeline	create_assessment, prediction_series, build_response, DISCLAIMER	views, reports tests
assessments/views.py	Assessment/history/trend/alert APIs	_resolve_patient + 5 views	urls
assessments/trends.py	Trend algorithm	compute_trend	services, selectors, reports
assessments/rules/engine.py	Deterministic rules	load_rules, evaluate, _OPS, CATEGORY_RANK	services, chat, serializers, tests
assessments/rules/rules.json	Rule data v1	21 rules, vocabulary, messages	engine
assessments/rules/keywords.py	Chat text -> symptoms	KEYWORD_MAP, detect_symptoms	chat services
mlcore/constants.py	ML contract	FEATURES, RISK_LEVELS(_ORDER), COLUMN_MAP, PLAUSIBILITY_RANGES, FEATURE_LABELS	everything ML + serializer
mlcore/dataset.py	Load/validate/clean	load_raw_dataset, validate_dataset, clean_dataset, feature_matrix, dataset_md5	training, inspect command, synthetic
mlcore/preprocessing.py	Median imputer	Preprocessor (fit/transform/to_dict/from_dict)	training, registry
mlcore/training.py	Train RF+XGB	train_all, TRAIN_CONFIG, model factories	command, tests
mlcore/evaluation.py	Metrics + selection	evaluate_classifier, selection_score	training, command
mlcore/registry.py	Serving	get_production_bundle, predict, feature_importance	assessments services, mlcore views
mlcore/explain.py	SHAP	explain_prediction (+fallback)	registry
mlcore/exceptions.py	503 error type	ModelUnavailableError	registry, services
mlcore/models.py	Model registry table	ModelVersion	command, views, registry
mlcore/views.py	Model APIs	list/compare/activate/importance/diagnostics	urls
mlcore/serializers.py	Read-only version serializer	—	views
mlcore/management/commands/*.py	train_models, inspect_dataset, generate_synthetic	CLI ops	operator
mlcore/synthetic.py	Research-only synthetic temporal data	generate_synthetic_temporal, quality_report	command
kb/models.py	Docs + chunks	KnowledgeDocument, DocumentChunk	services, views
kb/services/ingestion.py	Extract + chunk + sha256	extract_text (pypdf), chunk_text (900/150)	indexing
kb/services/embeddings.py	TF-IDF / OpenAI vectors	fit_corpus, embed_texts, get_backend	indexing, rag
kb/services/indexing.py	Index pipeline	index_document, _reembed_others	views
kb/services/store.py	Vector store + Qdrant mirror	retrieve, _retrieve_qdrant	rag
kb/services/rag.py	Retrieval + citations	retrieve_context (TOP_K=5, MIN_SCORE=0.05)	chat services, views
kb/views.py / serializers.py / urls.py / tests.py	KB APIs	upload/reindex/retrieve/chunks	config urls
chat/models.py	Sessions/messages	ChatSession, ChatMessage	services, views
chat/services.py	Safety pipeline	respond, _patient_context, CHAT_DISCLAIMER	SendMessageView
chat/llm.py	LLM client	SYSTEM_PROMPT, FALLBACK_NO_LLM, generate_answer, llm_configured	services
chat/views.py	Session/send APIs	serializers inline, _resolve_session	urls
reports/models.py / services.py / views.py / urls.py / tests.py	PDF reports	generate_report (ReportLab), ReportSerializer, download	config urls
audit/models.py / services.py / views.py / urls.py / serializers.py / admin.py	Audit trail	19 action constants, log_event with REDACTED_KEYS, admin-only list	every app
frontend/src/api/client.js	HTTP core	storage, interceptors, unwrap, apiError, api	all services
frontend/src/services/auth.js	Endpoint wrappers	8 service objects	all pages/panels
frontend/src/context/AuthContext.jsx	Auth state	AuthProvider, useAuth, forced-logout listener	guards, layout, login
frontend/src/App.jsx, main.jsx	Routing/boot	route tree, RoleDashboard	—
frontend/src/constants/clinical.js	Blood-group options	sync note w/ backend	patient forms
frontend/src/utils/reportDownload.js	Blob download	—	reports/history/overview

backend/scripts/smoke_test.py	E2E script	full workflow assertions	operator
scripts/dataset_profile.py	stdlib CSV profiler	md5, stats, dup check	analysis
backend/data/dataset_report.json	Persisted inspection report	6103 rows, cleaning stats, numeric summary	documentation/testing reference

35. Technology -> Why Map

Technology	Where used	Why used	What it provides
Django	whole backend	batteries-included framework: ORM/auth/admin/migrations	secure data layer + rapid CRUD
DRF	all APIs	serialization, auth pluggability, pagination, exception hooks	consistent JSON APIs
SimpleJWT	settings + accounts	stateless staff auth w/ blacklist	scalable API auth
django-cors-headers	middleware	SPA on 5173 <-> API on 8000	cross-origin dev
drf-spectacular	urls	auto OpenAPI + Swagger	living API docs
python-dotenv	settings	12-factor config	secret management
SQLite	default DB	zero-config dev	file DB; swap via DATABASE_URL
pandas/numpy	mlcore, kb store	data wrangling + vector math	ML pipeline
scikit-learn	training/eval/embeddings	RF, splits, metrics, TF-IDF	model 1 + retrieval
XGBoost	training	strong tabular booster	model 2 (current production per smoke test)
SHAP	explain	tree attributions	explainable predictions
joblib	persistence	bundle model+preprocessor+metadata	hot-swappable models
reportlab	reports	programmatic PDF	downloadable records
openai SDK	chat/kb	OpenAI-compatible HTTP client	LLM + optional embeddings
React 19	SPA	component UI	interactive staff console
Vite	tooling	instant dev server + proxy + build	DX
react-router 7	routing	nested layouts + guards	role-aware navigation
axios	client	interceptors (JWT/refresh)	resilient API calls
recharts	charts	trend/pie/bars/confusion visuals	data comprehension
oxlint	linting	hook-safety rules	code hygiene

36. Security Analysis

- Password hashing via Django's auth (set_password everywhere; passwords never serialized — UserSerializer fields exclude it).
- Password strength via AUTH_PASSWORD_VALIDATORS (similarity, length, common, numeric) + validate_password on change/create.
- JWT with short access (30 min), blacklistable refresh, role+active revalidation on refresh, generic 401 messages.
- Role-based permissions + object-level patient scoping reused across every patient-touching app; explicit patient parameter required on assessments/history/trends (_resolve_patient) so the frontend cannot silently target the wrong record.
- Structured errors that never leak stack traces (DEBUG-gated debug_trace only).
- Audit logging of security-relevant events (logins/failed logins, password change, assessments, escalations, KB/model/user/patient mutations) with REDACTED_KEYS (password, token, access, refresh, secret, ssn) and a "never store medical values or chat content" rule; audit is admin-only to read.
- ORM-only queries -> SQL injection effectively prevented (no raw SQL found anywhere).
- XSS: React escapes by default; no dangerouslySetInnerHTML anywhere (verified); chat renders text nodes.
- File-download authorization (ReportDownloadView checks access; test proves stranger gets 403).
- Input validation at three layers (UI min/max, serializer ranges/vocabulary, rule plausibility); chat message length cap (4000).
- Generic password-reset response (no user enumeration).
- Soft-delete users (audit history preserved).
- .gitignore excludes .env, db.sqlite3, media/, ml_artifacts/.

Implemented (verified in code):

1. The committed-looking API key in backend/.env.example — rotate/revoke and purge from history. Highest-priority item.
2. Tokens in localStorage (XSS-exfiltratable) — httpOnly cookies would be safer.
3. No rate limiting / throttling on login or chat (DRF throttles not configured).
4. No CSRF concerns for the SPA (JWT header auth), but the Django admin relies on session/CSRF — fine, just note the dual scheme.
5. pages/hcw/Reports.jsx direct /media/ link bypasses the authorized download endpoint and requires publicly served media in production.
6. pypdf/dj_database_url missing from requirements.txt.
7. Incomplete password-reset flow on the frontend. PasswordResetRequestView emails a link of the form /reset-password?token=<jwt>, but frontend/src/App.jsx defines no /reset-password route — the emailed link lands on the SPA wildcard redirect. The backend view's own docstring calls this "Password-reset request architecture" — i.e., the request half is implemented, the "set new password" half is not found in the inspected codebase. The service wrapper authService.requestPasswordReset exists but has no UI caller.
8. JWT is stateless and cannot be revoked per-token until expiry; mitigation present is the refresh blacklist (LogoutView), but a stolen access token remains valid up to 30 minutes. No token binding/IP pinning exists.
9. X_FRAME_OPTIONS/security headers rely on Django defaults (XFrameOptionsMiddleware is in the stack); no explicit SECURE_* settings (HSTS, secure cookies, SSL redirect) are configured — acceptable for dev, required for production.
10. Secrets in client bundle: VITE_API_BASE_URL is the only frontend env var — no secrets ship to the browser (good). All LLM/vector secrets stay

server-side (good).

11. Audit ip_address trusts X_FORWARDED_FOR first (audit/services.py) — spoofable unless a trusted proxy strips/sets it. Minor.
12. Model artifacts integrity: ModelVersion.dataset_version records the dataset md5 and train_models persists it, but nothing verifies artifact integrity at load time (a replaced .joblib would load silently). Low risk, worth noting for a defense answer.

Should be improved (gaps, stated honestly):

37. Error Handling

The project has an unusually deliberate, layered error strategy.

- core/exceptions.py — the contract. Every DRF error becomes {"success": false, "error": {"code","message","details"}} via structured_exception_handler:
- Http404 -> NOT_FOUND; Django PermissionDenied -> PERMISSION_DENIED.
- ApiError (raised by services/views) -> its own code/status_code/details — used pervasively (PERMISSION_DENIED 403, VALIDATION_ERROR 400, MODEL_UNAVAILABLE 503 via subclass).
- DRF ValidationError -> code VALIDATION_ERROR, details = the field-level dict (this is what the frontend maps onto form fields).
- NotAuthenticated/AuthenticationFailed -> AUTHENTICATION_ERROR.
- set_rollback() is called for API exceptions and unhandled exceptions — partial DB writes never persist.
- Unhandled exceptions -> logged with stack, client gets the opaque INTERNAL_ERROR 500 ("The issue has been logged").
- flatten_error_messages() walks nested DRF detail (dicts/lists/ErrorDetail) into one human sentence so "validation errors are never hidden behind a generic banner."
- core/middleware.py — the safety net. StructuredErrorMiddleware converts Django's HTML 404/500 pages on /api/* into the JSON envelope (so the SPA never receives HTML), and process_exception returns a 500 JSON — adding debug_trace only when DEBUG=True.
- Domain-level fail-safes:
- ModelUnavailableError(ApiError) -> 503 with a hint ("Ask an administrator to train and activate a model") — a prediction is never fabricated (mlcore/exceptions.py, registry.py).
- mlcore/explain.py: SHAP failure is caught, logged, and replaced by {"method":"unavailable","features":[]} — "Explainability must never take the prediction endpoint down."
- chat/llm.py: generate_answer raises nothing; any exception -> (None, "no_llm_configured").
- kb/services/rag.py: retrieve_context never raises — empty KB/failed embedding -> None.
- chat/services.respond: LLM unavailable -> rag_only_fallback (verbatim excerpts) or no_llm_configured message.
- assessments/services.create_assessment: transaction.atomic() wraps Assessment+Prediction+Alert — rule/prediction persistence is all-or-nothing.
- audit/services.log_event: wrapped in try/except — "audit must never break the request path."
- kb/views.perform_create: indexing ValueError -> document deleted -> clean 400 (no orphan rows).
- mlcore/views.ModelActivateView -> 404 ApiError for unknown model id; FeatureImportanceView -> 503 on bundle failure.
- chat/views.SendMessageView -> 400 for empty or >4000-char messages.

Backend layers

- api/client.js: single-flight 401 refresh-and-retry; on refresh failure clears tokens and broadcasts aimhra:logout (AuthContext clears user -> guards redirect to /login). apiError() normalizes backend envelopes and network failures (ERR_NETWORK -> "Cannot reach the server…").
- Component pattern: every data page renders Loading -> ErrorState (with Retry) -> EmptyState -> content. Forms map error.details onto per-field errors (PatientNew.jsx, ProfilePanel.jsx, AssessmentPanel.jsx). ChatPanel inserts a synthetic assistant bubble "Sorry - <message>" on failure instead of crashing the thread.

Frontend layers

Concrete failure traces

```
Invalid login -> 401 AUTHENTICATION_ERROR + LOGIN_FAILED audit -> Login.jsx red alert
Invalid assessment -> 400 VALIDATION_ERROR (field details) -> inline field errors; nothing saved
No production model -> 503 MODEL_UNAVAILABLE -> create_assessment aborts before ANY write
ML artifact corrupt -> 503 with artifact-path message (registry)
SHAP crash -> prediction still returns, explanation method="unavailable"
LLM down/unset -> rag_only_fallback or no_llm_configured reply (modes visible in UI)
LLM API exception -> logged, fallback answer — chat never 500s
Report download by stranger -> 403 PERMISSION_DENIED (proven by reports/tests.py)
Any unhandled error -> 500 JSON, stack only in server logs (debug_trace only in DEBUG)
```

38. Unused / Dead / Suspicious Code

Found by tracing imports and git status. Nothing was modified or deleted.

Item	Location	Finding
requests dependency	backend/requirements.txt line 15	Declared but not imported anywhere in the codebase — dead dependency
pypdf	kb/services/ingestion.py	Used but missing from requirements.txt (installed ad-hoc in the venv) — inverse problem
dj_database_url	config/settings.py line 84	Imported only when DATABASE_URL is postgres; not in requirements and not installed — a postgres DATABASE_URL would crash settings
ACTION_REGISTER	audit/models.py line 6	Dead action constant: the register endpoint was removed (smoke test asserts 404); still listed in AdminAudit.jsx filter options
Password-reset dead link	PasswordResetRequestView -> /reset-password?token=	No /reset-password route in App.jsx — emailed link has no landing page (incomplete feature, not dead code per se, but unreachable UX)
backend/admin.txt	repo root of backend	Empty file, purpose unclear

backend/full_test_out.txt, build_out.txt	stray, untracked	Test/build console output committed-adjacent clutter
backend/visualization.ipynb	untracked	Notebook not referenced by any code
pv1.zip	repo root	Unexplained archive, not referenced by code
Root Mathernal_Risk.csv	repo root	Duplicate of backend/data/Mathernal_Risk.csv; only consumer is scripts/dataset_profile.py (hardcoded absolute path to it)
/media/ direct link	pages/hcw/Reports.jsx line 37	Bypasses the authorized /api/reports/<id>/download/ endpoint used everywhere else — inconsistent and unsafe in production
Shared-bundle mutation	mlcore/registry.predict() lines 87-88	Writes _predicted_index/_predicted_label onto the cached, shared bundle dict; under concurrent requests two predictions could interleave before explain_prediction reads them — a real (if unlikely) race affecting the SHAP target class. Worth flagging in defense as a known simplification
Overstated docstring	kb/services/ingestion.py chunk_text	Docstring says markdown headings become section metadata, but no heading extraction is implemented; DocumentChunk.section stays empty
Envelope inconsistency	several list endpoints	PatientListView/AssessmentListCreateView return manual {results,count} even though StandardPagination paginated the queryset (first 20 only, no page param sent by the FE); reports/chat return {reports}/{sessions} — the frontend's defensive d.results ?? d patterns absorb it, but the API surface is inconsistent
First-page-only lists	Patients.jsx, Dashboard.jsx, Alerts.jsx	They fetch default page 1 (20 items) and sort client-side — with >20 records the UI silently shows a subset (no pager except AdminAudit.jsx)
IsAdminOrReadOnly	core/permissions.py	Used only by the kb app — fine, but effectively single-consumer
Deleted-but-tracked files	git status	frontend/src/pages/patient/* (7 files) and Register.jsx deleted in the working tree — the role refactor; current App.jsx no longer references them (clean removal)

No TODO/FIXME markers, no console.log/print debug statements in app code, and no hardcoded credentials were found other than the .env.example API key already flagged in §28/§36.

39. Strengths

1. True separation of concerns — views (HTTP) -> serializers (validation) -> services/selectors (logic) -> registry (artifacts). assessments/services.py and patients/selectors.py are the proof; no view duplicates access logic.
2. Train/serve preprocessing parity by construction — the fitted Preprocessor is serialized inside the joblib bundle, and the serializer validates against the same PLAUSIBILITY_RANGES used to clean training data. This eliminates the classic train/serve skew bug class.
3. Explainability as a first-class citizen — SHAP computed per prediction, persisted on Prediction.explanation, rendered in UI and PDF, with an anti-causation disclaimer embedded in the payload itself.
4. Deterministic, versioned safety layer that outranks everything — rules.json v1 is data (inspectable by clinicians), stamped onto every Alert/ChatMessage, and the chat pipeline hard-bypasses the LLM on escalation (RULE_ESCALATION audited).
5. Fail-safe defaults everywhere — 503 instead of fabricated predictions; SHAP/chat/RAG all degrade gracefully; audit and SHAP failures can never take down the request path.
6. Honest scope boundaries — "does not predict future risk" is enforced in trends.py, echoed in API responses, the UI, and the PDF, and asserted by tests.
7. Model governance — ModelVersion registry (dataset md5, hyperparameters, metrics, selection criterion, CANDIDATE/PRODUCTION/RETIRED lifecycle), predictions pinned to the exact version that produced them (model_ref), admin-only activation with audit.
8. Centralized access control — one can_access_patient / patient_ids_for_healthcare_worker implementation reused by patients, assessments, alerts, reports, and chat.
9. Consistent API envelope + resilient client — {success,data|error} everywhere; the axios client handles attach/refresh/blacklist-logout in one place, with single-flight refresh.
10. Test depth for a student project — cross-worker isolation (403 with no row created), legacy-role rejection at three layers, 503-no-fabrication, rule engine unit tests, trend unit tests, RAG citation tests, PDF content tests (%PDF magic bytes), plus a runnable E2E smoke script.
11. Research hygiene — mlcore/synthetic.py is explicitly quarantined: always flagged is_synthetic, quality-gated, documented as "not TimeGAN", never mixed into production training.
12. Accessibility-minded UI — color is never the sole risk indicator (icon + text always), stated in both ui.jsx and styles.css comments.

40. Weaknesses

1. Secret hygiene — the apparent real API key in .env.example (§28) is the single most serious issue; also no CI/lint gate preventing recurrence.
2. Auth token storage in localStorage and no throttling on /auth/login/, /auth/refresh/, or chat (§36).
3. Dependency management — unused requests, missing pypdf/dj_database_url; no lockfile for Python; loose version ranges.
4. Frontend pagination blind spot — patient/assessment/alert lists fetch only page 1 (20 items) and sort client-side; no pager UI outside the audit page (§38).
5. Inconsistent response envelopes across list endpoints force defensive parsing in the frontend (§38).
6. RAG scalability — every upload re-fits the corpus-wide TF-IDF vectorizer and re-embeds all other documents, and every retrieval loads all chunk vectors into memory. Fine for dozens of documents; documented by the author as a known trade-off.
7. predict() shared-bundle race (§38) — a correctness smell under concurrency, easy to fix by passing the predicted index explicitly.
8. /media/ report link in one page bypasses authorization (§38) and depends on DEBUG media serving.
9. Duplication between FE and FE validation — AssessmentPanel.FIELDS min/max must be hand-synced with PLAUSIBILITY_RANGES; clinical.js carries an explicit "keep in sync" comment — a recognized maintenance tax.
10. No deployment story — no Dockerfile, no gunicorn/nginx config, no CI workflow anywhere in the repo; SQLite default; media served only in DEBUG.
11. Documentation — root README.md contains only "# AIMHRA"; frontend README is Vite template boilerplate. (Docstrings inside code are, however, excellent.)
12. Dead/inert artifacts — admin.txt, full_test_out.txt, build_out.txt, pv1.zip, duplicate root CSV (§38).

13. Password-reset flow incomplete on the frontend (§36 item 7).
14. Test gaps — no frontend tests, no reset-email tests, no throttle/security tests; tests depend on quick models so model-quality regressions aren't caught.

41. Limitations (including medical/safety boundaries)

Claim	NOT true, and here's the proof it's prevented
"The risk level is a diagnosis"	assessments/services.DISCLAIMER: "not a medical diagnosis and does not replace professional medical assessment" — returned in every assessment response and printed in the PDF's Disclaimer section; repeated by the UI Disclaimer component
"The trend predicts future risk"	trends.py docstring: no temporal model, dataset has no visit fields; RiskTrendView response ships the literal string "The system does not predict future risk"; PDF and both chart panels repeat it; a test asserts it
"The AI assistant is a doctor"	chat/llm.SYSTEM_PROMPT forbids diagnosis/prescription/exam claims; CHAT_DISCLAIMER on every reply; escalation answers state "This chat assistant cannot provide emergency care"
"SHAP shows medical causes"	explain.EXPLANATION_DISCLAIMER: "statistical contributions, not medical causes"; wording rule enforced in explain.py and the PDF
"Rule thresholds are clinically certified"	rules.json note: thresholds are configurable defaults that "MUST be reviewed and approved by qualified clinical stakeholders before any real-world deployment"
"The model knows the patient's history/demographics"	The 8 model features are only the assessment measurements; PatientProfile docstring states demographics are "never fed to the ML model"
"Patients use the system"	Patients are non-authenticating records; legacy PATIENT accounts are deactivated by migration and rejected at login/refresh
"The data is clean/consistent"	constants.DATASET_NOTES documents the glucose unit inconsistency as a recorded limitation, values used as-is

Other factual limitations: single-dataset training (6,103 rows, one source); production model chosen by high_risk_recall_then_f1 on one 15% test split; local vector store only by default (Qdrant optional and not installed); chat depends on an external LLM whose absence degrades to excerpts; LLM output is not programmatically verified against sources (grounding is prompt-enforced, not enforced in code); no scheduling/notifications (alerts are pull-based — there is no email/SMS notification system in the codebase).

42. How to Explain This Project in a Viva

Q: What is the project?

A: A role-based maternal-health decision-support system. Healthcare workers record patient visits; an ML model classifies risk (low/mid/high) with probabilities; SHAP explains it; an independent rule engine raises safety alerts; history, trends, PDF reports, and a RAG-grounded assistant complete the workflow. Two staff roles only — healthcare worker and admin; patients are records, not accounts.

Q: Why Django?

A: It gives, in one framework, the ORM, an auth system extended with a role field (accounts.User), migrations for schema evolution, an admin for oversight, and — via DRF — clean JSON APIs with pluggable authentication. The rule engine, ML registry, and audit service all plug in as ordinary Django apps, which keeps the safety layer inside the same transactional boundary as the data.

Q: Why React?

A: The staff console is highly interactive — tabbed patient workspaces, live forms with per-field errors, charts. React's component model maps directly onto that (PatientDetail.jsx with seven panels), and its ecosystem gives routing (react-router-dom) and charting (recharts) without a page reload.

Q: Why a REST API (separate frontend/backend)?

A: It forces the safety-critical logic — access control, validation, prediction, rules — to live server-side where it can't be bypassed, and lets the SPA stay thin. The uniform {success, data|error} envelope makes one axios client handle auth, refresh, and errors for every endpoint.

Q: Why JWT?

A: Stateless authentication fits an API: each request carries a short-lived access token (30 min), and the refresh token (7 days) is blacklisted at logout. The custom refresh serializer re-checks role and is_active from the database on every refresh, so deactivated or legacy accounts can't mint new sessions.

Q: Why XGBoost and Random Forest?

A: Both are strong tabular classifiers that need no feature scaling, handle non-linear clinical thresholds, and expose predict_proba and feature_importances_. Both are trained on identical splits and a criterion — high-risk recall first, then macro-F1 — picks production, because a missed high-risk case is the costliest error. Accuracy alone is never the selection basis.

Q: Why SHAP?

A: Tree ensembles are black boxes; in a clinical support tool that's unacceptable. shap.TreeExplainer gives per-feature contributions for the exact predicted class, persisted with the prediction so the explanation is reproducible; the UI draws positive contributions pushing risk up, negative pulling it down — always labeled as statistical contributions, not causes.

Q: Why rule-based alerts in addition to ML?

A: The model is probabilistic; emergencies must be deterministic. A systolic BP >= 160 or reported convulsions must always escalate, even if the model says low risk. Keeping rules in a versioned JSON file makes them inspectable by clinicians and auditable — every outcome carries rules_version.

Q: Why three risk levels?

A: Because the dataset's target has exactly three classes — low 2001 / mid 2043 / high 2059 — and inventing other labels would break train/serve consistency. The three levels are the canonical contract in mlcore.constants.RISK_LEVELS.

Q: Why a database at all (why not just compute)?

A: Because the clinical value is longitudinal: each visit's inputs and the exact model output are stored (Assessment, Prediction with model_ref), which is what makes history, trends, reports, alerts, and audit possible — and it lets every prediction be traced to the model version that made it.

Q: Why RAG?

A: So the assistant answers from trusted, cited documents instead of the LLM's memory. Retrieval returns top-5 chunks above a minimum score;

the LLM is prompt-constrained to ground its answer in them; sources are stored with the message and rendered as citations.

Q: Why a vector store / embeddings?

A: To match a question to document passages by meaning. Chunks are embedded (TF-IDF locally by default, OpenAI embeddings optionally) and ranked by cosine similarity; the local backend stores vectors on the chunk rows so there's zero infrastructure, with a Qdrant collection available as a swap-in.

Q: Why an AI assistant at all?

A: For education and navigation — "what does this warning mean", "what should I ask the doctor". The pipeline checks the emergency rule engine first and bypasses the LLM entirely when it fires, so the assistant can never talk a user out of seeking care.

Q: Why PDF reports?

A: Assessments must be shareable and archival outside the software. ReportLab on the backend means one source of truth — the stored prediction, SHAP, trend, and alerts — and the download endpoint re-checks authorization per request.

Q: Why role-based access?

A: Least privilege. Workers see only patients they created or were assigned; admins manage users, models, the knowledge base, and audit. The same selector backs every patient-touching endpoint, and tests prove cross-worker access returns 403 with no data leak.

Q: What's innovative?

A: The combination: explainable ML (SHAP persisted per prediction), a deterministic rule engine that provably outranks both the model and the LLM, honest descriptive-only trends, versioned model governance, an audit trail with redaction, and graceful degradation everywhere (503 instead of fabricated predictions, excerpt-fallback chat).

Q: Limitations?

A: Single training dataset; unit inconsistency in the glucose columns documented but not fixed; no rate limiting; JWT in localStorage; local vector store loads everything into memory; trends use only the last two visits; no frontend tests; no deployment configuration.

Q: Future work?

A: Rotate and purge the exposed key; httpOnly-cookie auth; throttling; a proper temporal dataset to enable genuine longitudinal modeling; Qdrant for scale; background task queue for KB re-indexing; end-to-end frontend tests; Dockerized deployment with Postgres; clinical review and sign-off of rule thresholds.

43. Viva / Defense Question Bank (with answers grounded in this repo)

1. What does AIMHRA stand for? — "Hybrid AI-Based Maternal Health Risk Analysis and Healthcare Decision-Support System" (config/settings.py docstring / Spectacular description).
2. Who are the users? — HEALTHCARE_WORKER and ADMIN only; patients never log in (accounts/models.py, patients/models.py docstring).
3. What happens on an emergency finding? — Rule engine returns EMERGENCY, an Alert row is created, a RULE_ESCALATION audit entry is written, the UI shows a red banner; in chat the LLM is bypassed entirely.
4. How many rules exist? — 21 in rules.json v1 across four categories; each has an id, category, type (symptom/vital), and description.

Project Overview

1. Walk a request through the system. — Vite proxy -> Django URL -> DRF auth (JWT) -> permission class -> view -> serializer validation -> service -> ORM -> engine (ML/rules) -> ok()/error envelope -> axios unwrap() -> React state.
2. Where is business logic? — Services/selectors (assessments/services.py, patients/selectors.py, reports/services.py, chat/services.py, kb/services/*), not in views.
3. What is core/ for? — Cross-cutting infrastructure: permissions, the error contract, error middleware, pagination, success helper.

Architecture

1. Why a custom exception handler? — To guarantee one error shape and to map codes; flatten_error_messages makes field errors readable; set_rollback protects data.
2. How does pagination work? — StandardPagination (20/page, page_size query param, max 100) wrapping results in the envelope.
3. What does drf-spectacular provide? — Auto OpenAPI schema at /api/schema/ and Swagger UI at /api/docs/.

Django / DRF

1. How does the app restore a session on reload? — AuthContext sees a stored access token and calls GET /api/auth/profile/; failure clears tokens.
2. What happens when a 401 arrives mid-session? — The axios response interceptor refreshes once (single-flight), retries the original request; on failure it clears tokens and dispatches aimhra:logout, which the context listens for.
3. How is route protection done? — ProtectedRoute checks boot state, user existence, the two valid roles, then an optional roles array; wrong role redirects to the user's own dashboard.

React

1. Why is Prediction separate from Assessment? — One visit, one derived output, pinned to model_ref; keeps raw inputs immutable and model outputs versioned.
2. Why SET_NULL on many FKs? — Preserves clinical history even if the acting user/creator/model row is removed.
3. What do migrations 0002 do? — The role refactor: PATIENT choices removed, legacy accounts deactivated, patient identity copied off the user onto the standalone record and the user FK dropped.

Database

1. Where is the role check for login? — CustomTokenObtainPairSerializer.validate rejects roles outside HCW/ADMIN; CustomTokenRefreshSerializer repeats it at refresh time.
2. How is logout real if JWT is stateless? — The refresh token is blacklisted (LogoutView); the access token simply expires within 30 minutes.

Authentication

1. Describe the split and why. — Stratified 70/15/15 with random_state=42; stratification preserves the near-balanced class mix; metrics come only from the test split.
2. What preprocessing exists? — Median imputation fitted on the training split only, persisted inside the artifact; no scaling (trees don't need it).

3. What if someone posts age = 300? — Serializer range check -> 400 VALIDATION_ERROR; the same ranges removed implausible training rows.
4. How is the production model chosen? — selection_score with MODEL_SELECTION_CRITERION (default high-risk-recall -> F1 -> AUC); train_models retires other versions and marks the winner PRODUCTION.
5. What if no model is registered? — ModelUnavailableError -> HTTP 503 MODEL_UNAVAILABLE; the assessment is not saved (asserted by tests).

ML

1. Key hyperparameters? — RF: 400 trees, min_samples_leaf=2, class_weight="balanced_subsample". XGB: 400 rounds, lr 0.08, depth 6, subsample 0.9, colsample 0.8, hist, mlogloss.
2. Why class weights if the data is balanced? — Near-balanced (33/33/33) so resampling was rejected as synthetic-noise risk; weights kept as a safeguard, recorded in TRAIN_CONFIG.

XGBoost / Random Forest specifics

1. Which model is explained, and when? — The production bundle's model, right after predict_proba, on the same preprocessed vector; top-8 absolute contributions returned.
2. What do positive SHAP values mean here? — The feature pushed the predicted class probability up (red bars); negative pushed it down (green bars) — a model-level statement, never a causal claim.

SHAP

1. Can the ML model hide an emergency? — No: rules run independently of the prediction; EMERGENCY creates its own Alert and, in chat, bypasses the LLM. "Nothing downstream may soften an emergency" (engine.py).
2. How would you add a rule? — Add a row to rules.json (id/category/type/threshold) — the engine hot-reloads on mtime change; bump the version so outcomes are traceable.
3. Alert lifecycle? — OPEN -> ACKNOWLEDGED -> RESOLVED via /api/assessments/alerts/<id>/ack/, recording acknowledged_by.

Rules / Alerts

1. Chunking strategy? — 900 characters with 150 overlap; empty text yields no chunks and indexing fails with a clean 400.
2. How is hallucination mitigated? — Retrieval-grounded system prompt ("do not invent facts beyond them"), cautious-language instructions, <=250 words, sources stored and displayed; fallback answers use verbatim excerpts when the LLM is unavailable.
3. What is the default embedding and vector store? — Local TF-IDF (5000 features, English stop-words) with cosine similarity in numpy over JSON-stored vectors; Qdrant (aimhra_kb collection) is optional and not installed.
4. What if the knowledge base is empty? — retrieve_context returns None; chat replies that no indexed knowledge matched — no fabricated sources (tested).
5. Which LLM, exactly? — Any OpenAI-compatible endpoint; default configuration gpt-4o-mini via the openai SDK, temperature 0.3, max 600 tokens, 45s timeout.

RAG / LLM / Vector store

1. Where could SQL injection occur? — Nowhere: all queries go through the ORM; no raw SQL exists in the codebase.
2. How are downloads protected? — /api/reports/<id>/download/ re-checks can_access_patient and streams a FileResponse; a stranger's 403 is unit-tested.
3. What's the weakest security point? — The exposed-looking key in .env.example (rotate/scrub), tokens in localStorage, and no login throttling.

Security

1. How do tests get a model without long training? — train_all(..., quick=True) trains 10-tree models; only full training registers production versions.
2. How would you deploy this? — Postgres via DATABASE_URL, gunicorn behind nginx, DEBUG=False, real SECRET_KEY, media on object storage/CDN, collectstatic, train_models at deploy, HTTPS + HSTS — none of which is configured in the repo yet.

Testing / Deployment

44. Final Complete System Summary

AIMHRA is a staff-facing maternal-risk decision-support platform whose defining quality is defensive engineering: every stage of the pipeline has an explicit failure mode, an explicit disclaimer, and an explicit audit trail.

- Identity & access: custom User with two roles; JWT access (30 min) + blacklisted refresh (7 days); role re-validated at login and refresh; object-level patient access via one shared selector; legacy PATIENT role removed by migration and blocked at three layers.
- Clinical workflow: standalone patient records (created-or-assigned visibility) -> visit assessments capturing exactly 8 model features plus a fixed symptom vocabulary -> serializer validation mirroring training-data plausibility ranges -> atomic persistence of Assessment + Prediction + Alert.
- Intelligence: Random Forest and XGBoost trained on a documented, cleaned 6,057-row dataset (stratified 70/15/15, median-imputation preprocessor persisted in the artifact); production selection by high-risk-recall-first criterion; a versioned ModelVersion registry; every prediction stamped with the exact model name/version and a persisted SHAP top-8 explanation with an anti-causation disclaimer.
- Safety: a deterministic, versioned 21-rule engine (NORMAL->ATTENTION->HIGH_CONCERN->EMERGENCY) that outranks the model, RAG, and LLM; alerts with a full acknowledge/resolve lifecycle; chat messages scanned lexically for emergency phrases with LLM-bypassing escalation.
- Longitudinal view: history and trend computed only over stored predictions (last-two-visit comparison -> improving/stable/increasing), explicitly and repeatedly labeled as non-predictive.
- Knowledge & assistance: admin-curated RAG pipeline (PDF/TXT ingestion -> 900/150 chunking -> TF-IDF or OpenAI embeddings -> cosine top-5 retrieval with score floor -> cited, prompt-constrained OpenAI-compatible generation with excerpt fallbacks).
- Output & oversight: ReportLab PDFs generated and authorization-checked server-side; an admin console for users, assignments, model comparison/activation, knowledge management, and system stats; an admin-only audit log of 19 action types with key redaction.
- Quality posture: layered tests across auth, scoping, ML serving, rules, trends, RAG, chat safety, and reports, plus a runnable end-to-end smoke script; structured error envelope end-to-end; graceful degradation at every external dependency (model, SHAP, LLM, KB).
- Known debts, ranked: the exposed-looking API key in .env.example (rotate/scrub immediately); missing pypdf dependency; token storage and

throttling; the unauthorized /media/ report link; the incomplete password-reset landing route; first-page-only lists in the UI; the shared-bundle mutation race in predict(); and the absent deployment configuration. None of these undermine the architecture — all are addressable without structural change.

After reading this report, every layer of the repository — from rules.json thresholds to the axios refresh interceptor, from the TF-IDF refit strategy to the RISK_LEVEL_ORDER trend arithmetic — can be opened, located, explained, and defended.